

L6: Support Vector Machines

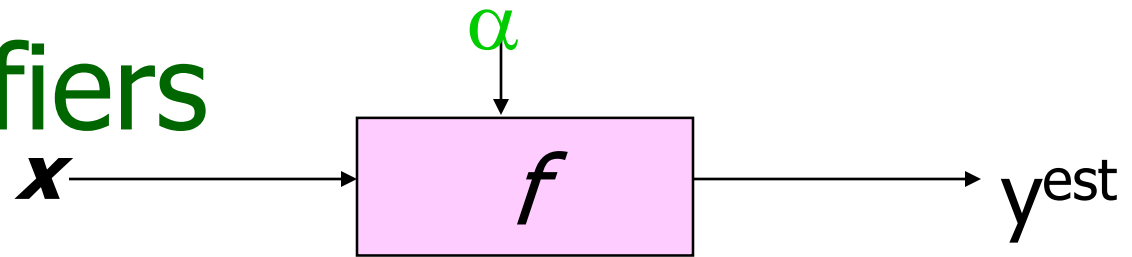
Modified from Prof. Andrew W. Moore's Lecture Notes

www.cs.cmu.edu/~awm

History

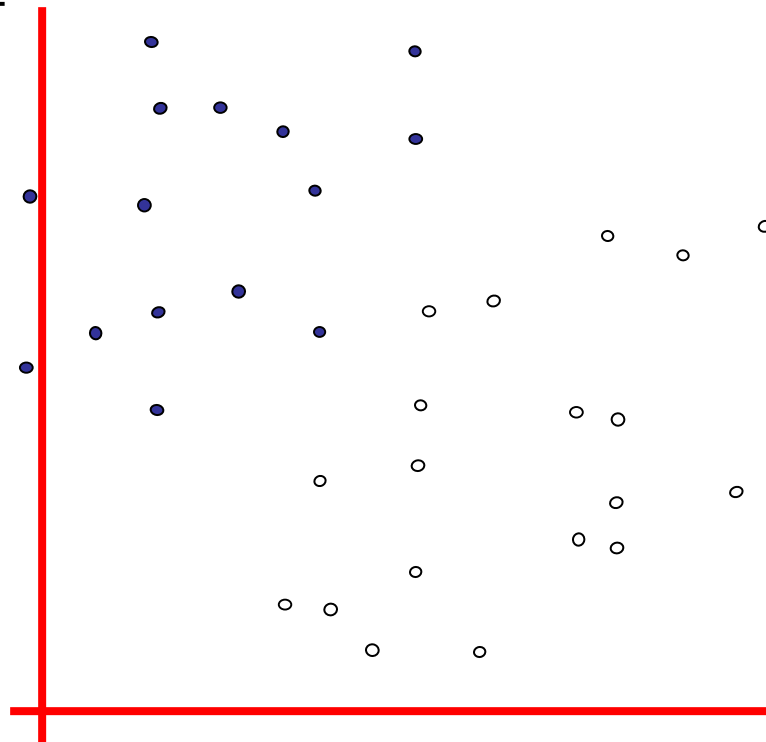
- SVM is a classifier derived from statistical learning theory by Vapnik and Chervonenkis
- SVMs introduced by Boser, Guyon, Vapnik in COLT-92
- Now an important and active field of all Machine Learning research.
- Special issues of Machine Learning Journal, and Journal of Machine Learning Research.

Linear Classifiers



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

- denotes +1
- denotes -1

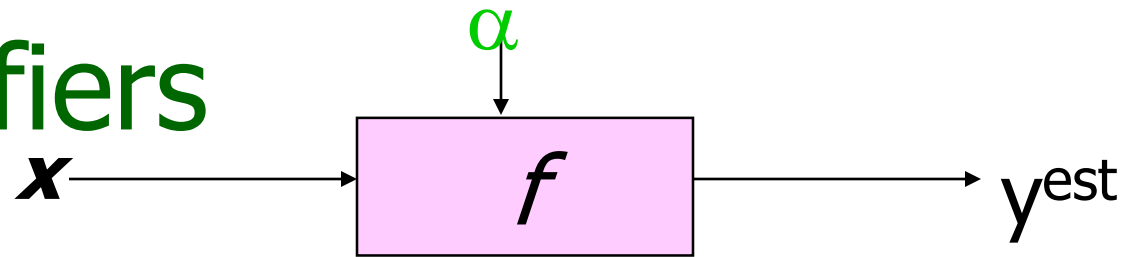


How would you classify this data?

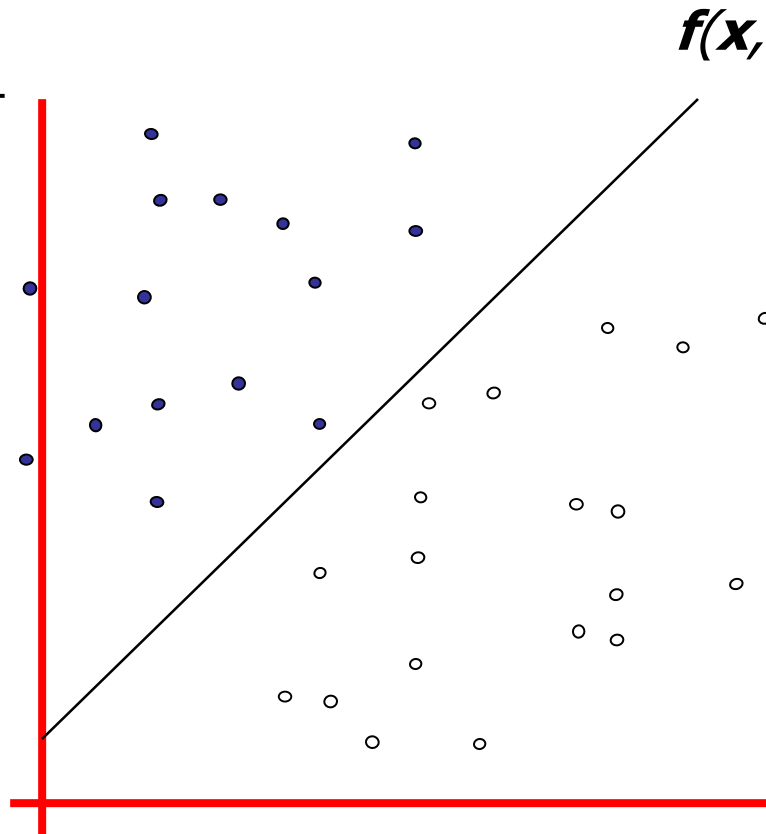
e.g. $\mathbf{x} = (x_1, x_2)$, $\mathbf{w} = (w_1, w_2)$, $\mathbf{w} \cdot \mathbf{x} = x_1 w_1 + w_2 x_2$

$\text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = +1$ iff $x_1 w_1 + w_2 x_2 - b > 0$

Linear Classifiers



- denotes +1
- denotes -1

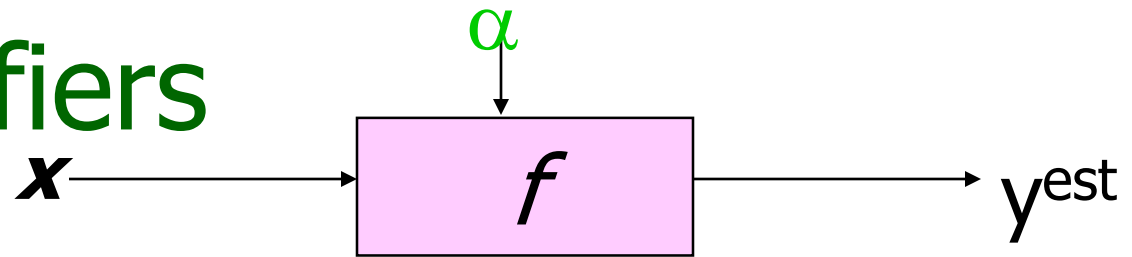


How would you classify this data?

e.g. $\mathbf{x} = (x_1, x_2)$, $\mathbf{w} = (w_1, w_2)$, $\mathbf{w} \cdot \mathbf{x} = x_1 w_1 + w_2 x_2$

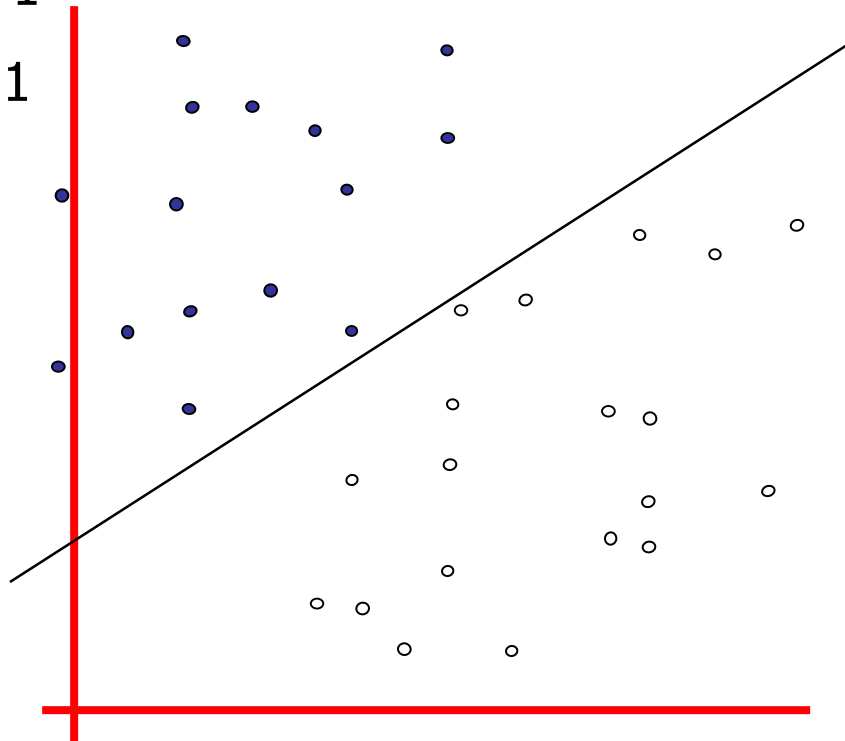
$\text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = +1$ iff $x_1 w_1 + w_2 x_2 - b > 0$

Linear Classifiers



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

- denotes +1
- denotes -1

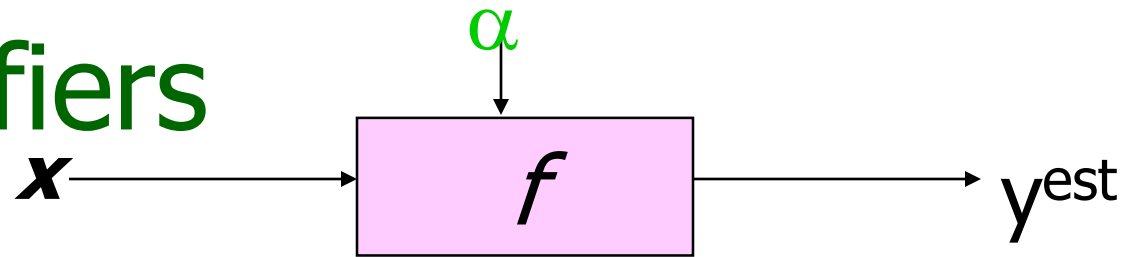


How would you classify this data?

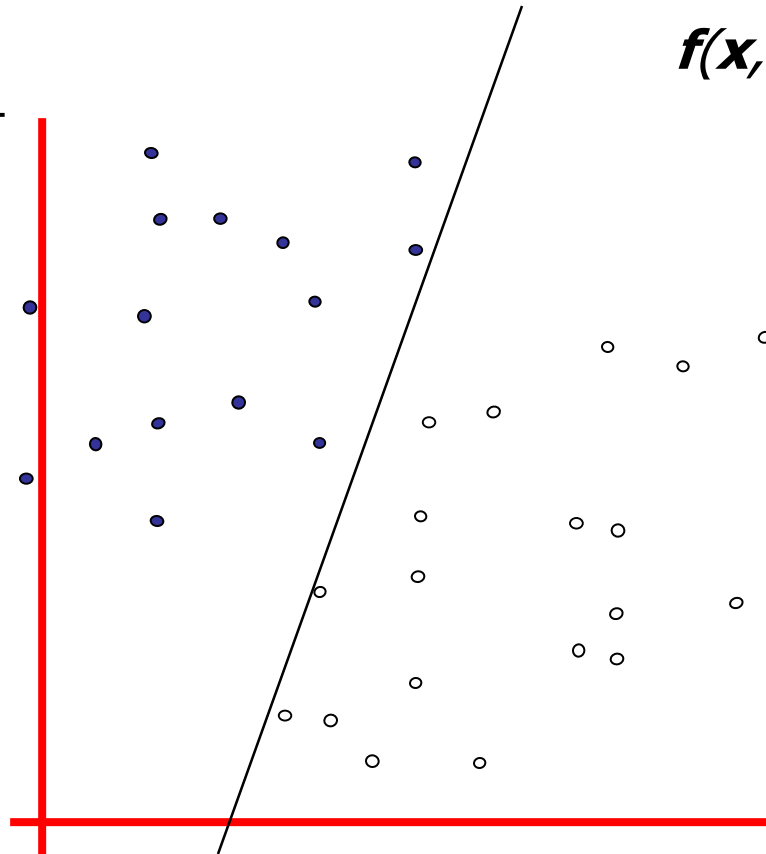
e.g. $\mathbf{x} = (x_1, x_2)$, $\mathbf{w} = (w_1, w_2)$, $\mathbf{w} \cdot \mathbf{x} = x_1 w_1 + w_2 x_2$

$\text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = +1$ iff $x_1 w_1 + w_2 x_2 - b > 0$

Linear Classifiers



- denotes +1
- denotes -1



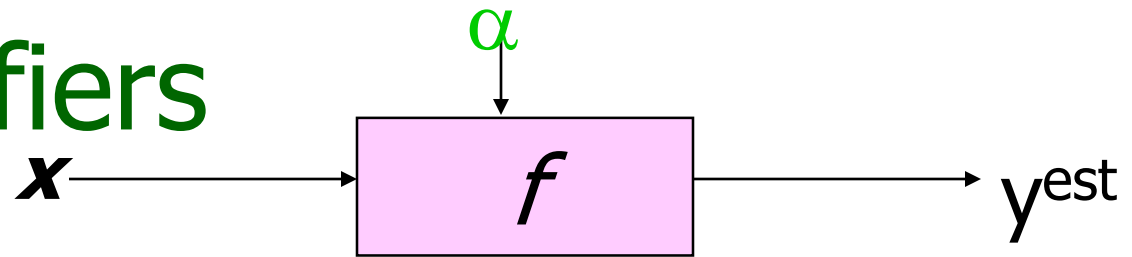
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

How would you classify this data?

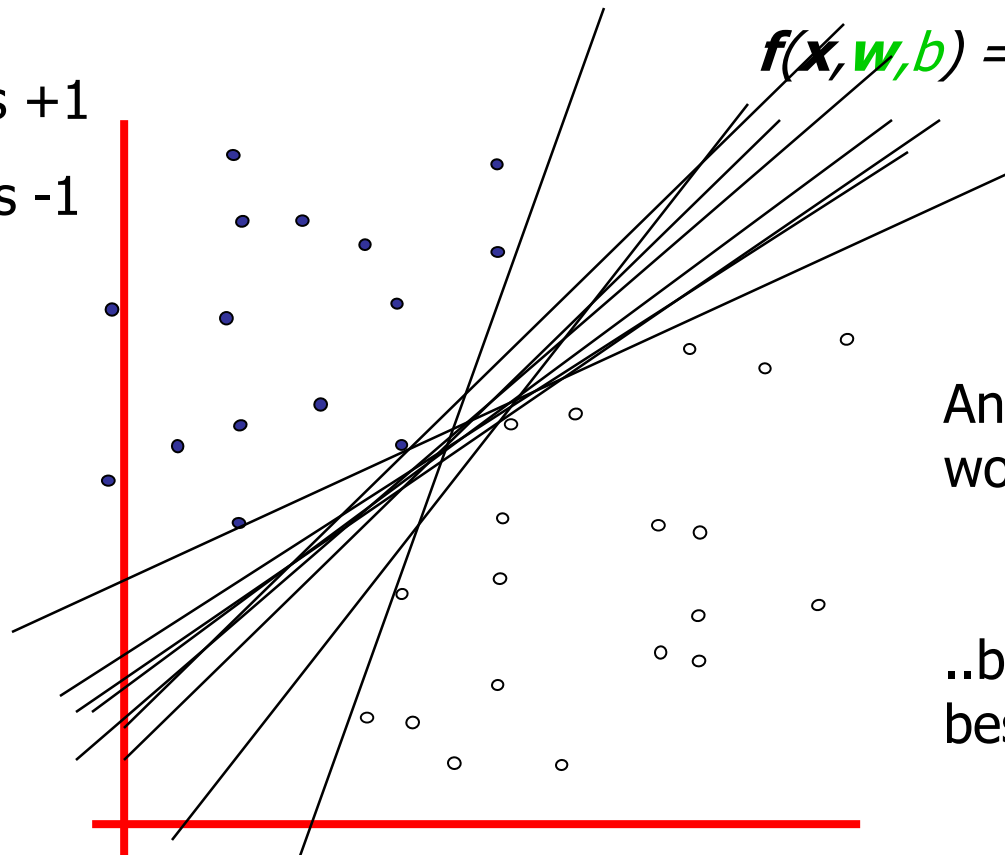
e.g. $\mathbf{x} = (x_1, x_2)$, $\mathbf{w} = (w_1, w_2)$, $\mathbf{w} \cdot \mathbf{x} = x_1 w_1 + w_2 x_2$

$\text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = +1$ iff $x_1 w_1 + w_2 x_2 - b > 0$

Linear Classifiers



- denotes +1
- denotes -1



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

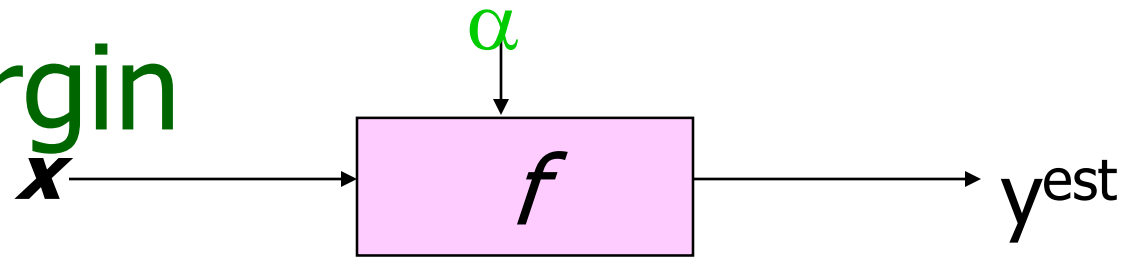
Any of these
would be fine..

..but which is
best?

e.g. $\mathbf{x} = (x_1, x_2)$, $\mathbf{w} = (w_1, w_2)$, $\mathbf{w} \cdot \mathbf{x} = x_1 w_1 + w_2 x_2$

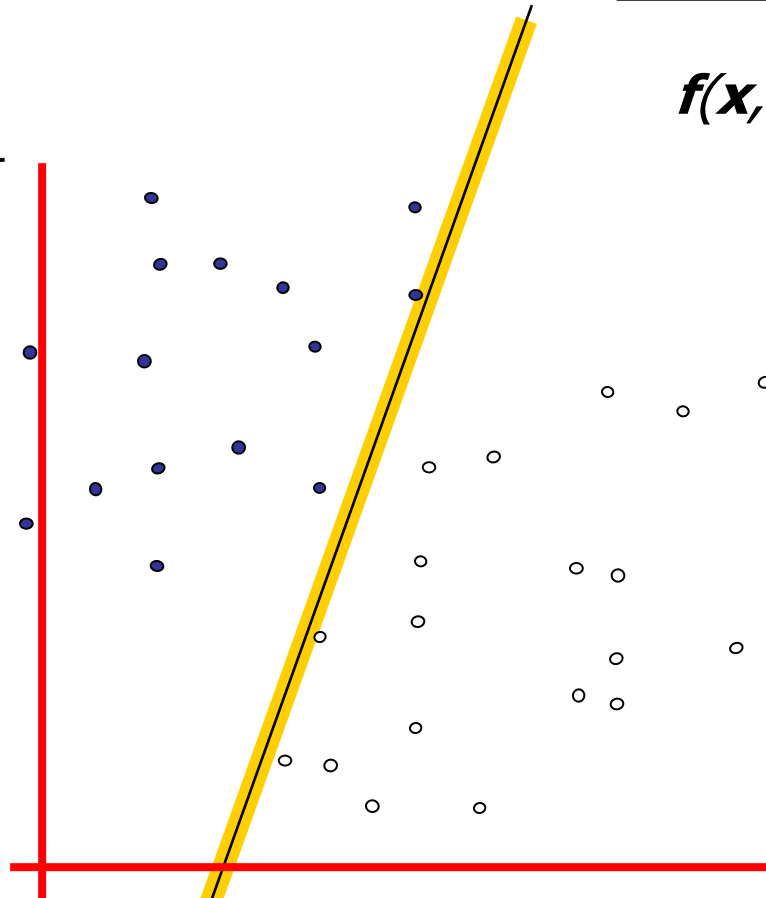
$\text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = +1$ iff $x_1 w_1 + w_2 x_2 + b > 0$

Classifier Margin



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

- denotes +1
- denotes -1

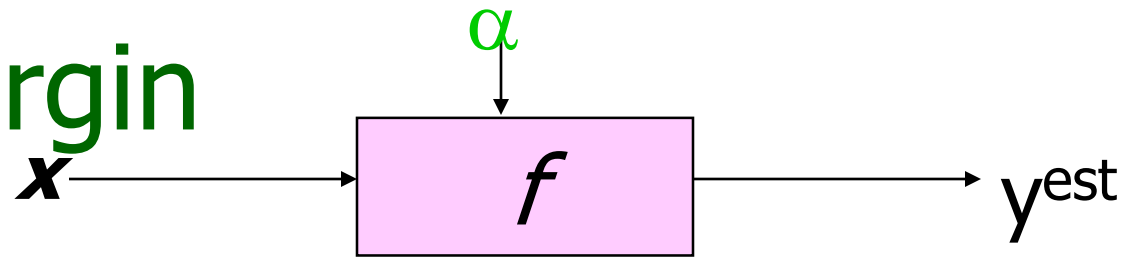


Define the **margin** of a linear classifier as the width that the boundary could be increased by before hitting a datapoint.

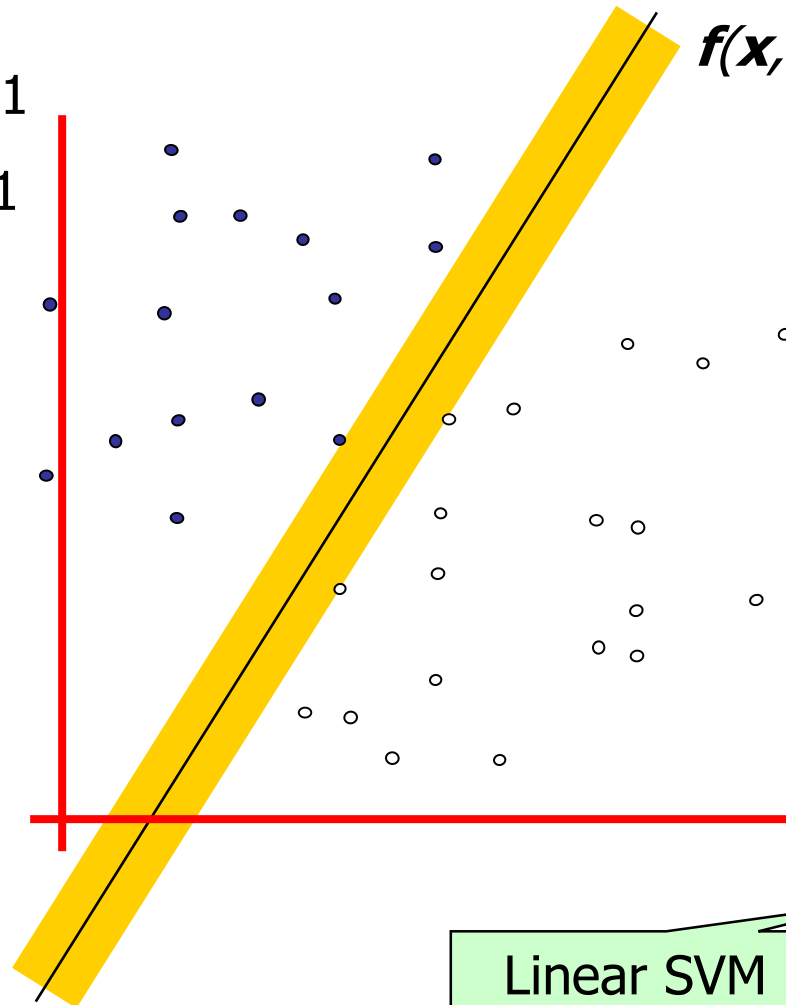
e.g. $\mathbf{x} = (x_1, x_2)$, $\mathbf{w} = (w_1, w_2)$, $\mathbf{w} \cdot \mathbf{x} = x_1 w_1 + w_2 x_2$

$$\text{sign}(\mathbf{w} \cdot \mathbf{x} + b) = +1 \quad \text{iff} \quad x_1 w_1 + w_2 x_2 - b > 0$$

Maximum Margin



- denotes +1
- denotes -1



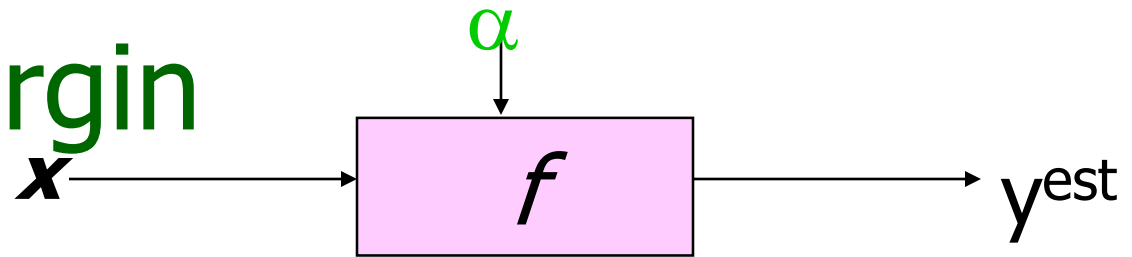
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

This is the simplest kind of SVM (Called an LSVM)

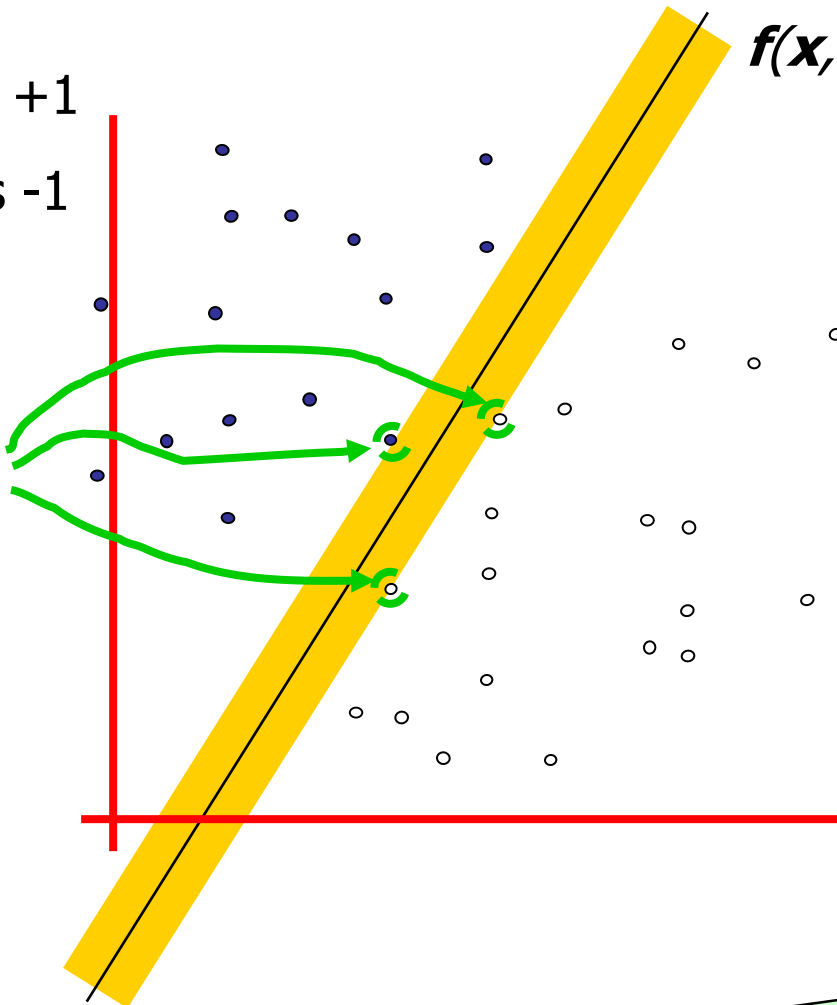
Linear SVM

Maximum Margin



- denotes +1
- denotes -1

Support Vectors
are those
datapoints that
the margin
pushes up
against



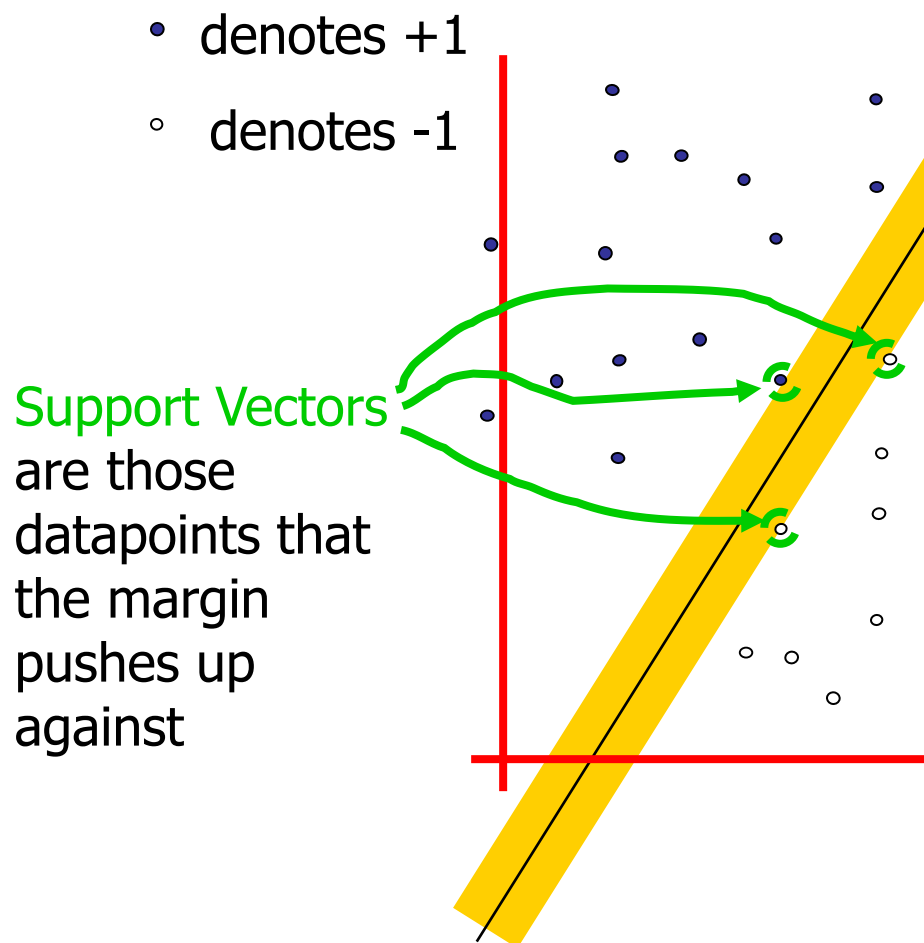
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

The **maximum margin linear classifier** is the linear classifier with the maximum margin.

This is the simplest kind of SVM (Called an LSVM)

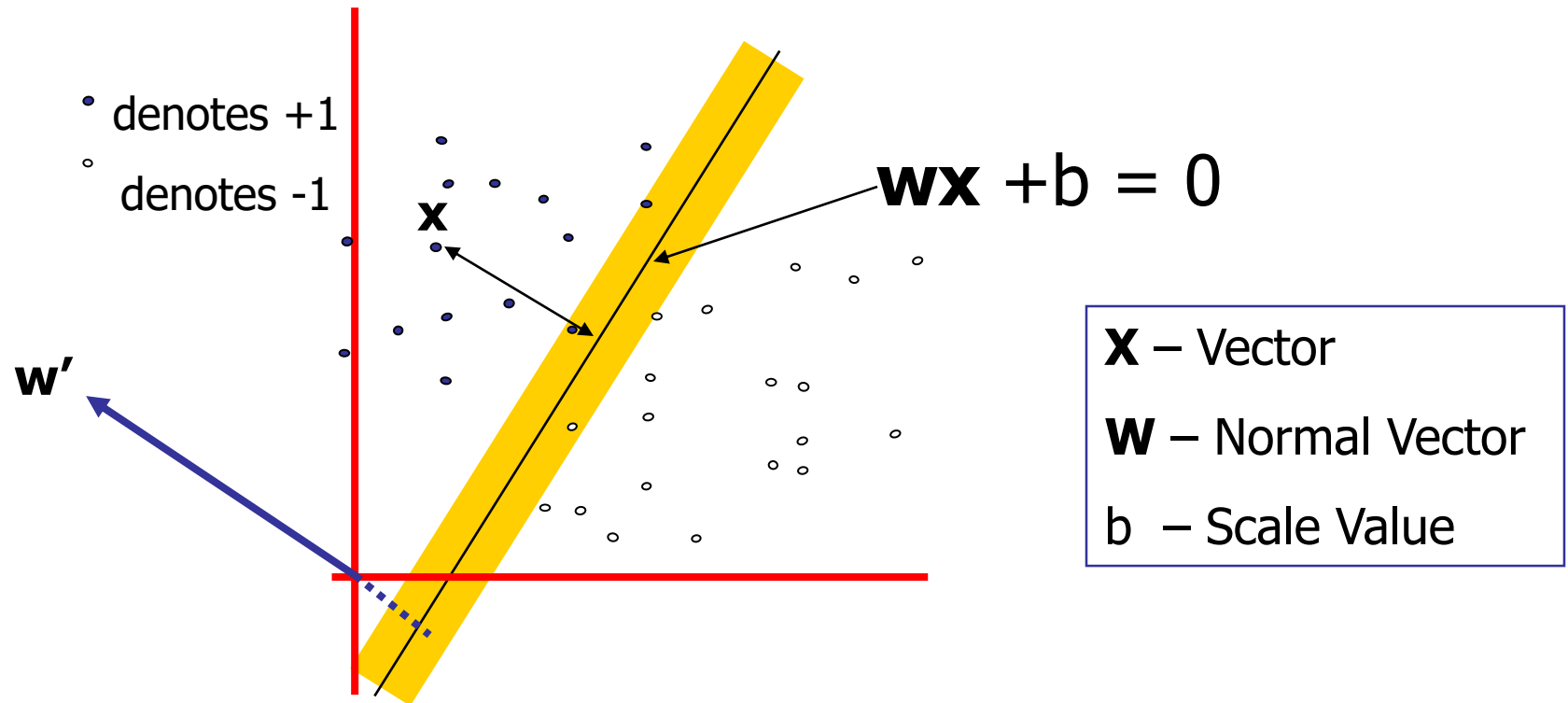
Linear SVM

Why Maximum Margin?



1. **Intuitively this feels safest.**
2. **Empirically it works very well.**
3. If we've made a small error in the location of the boundary (it's been jolted in its perpendicular direction) this gives us least chance of causing a misclassification.
4. There's some theory (using VC dimension) that is related to (but not the same as) the proposition that this is a good thing.

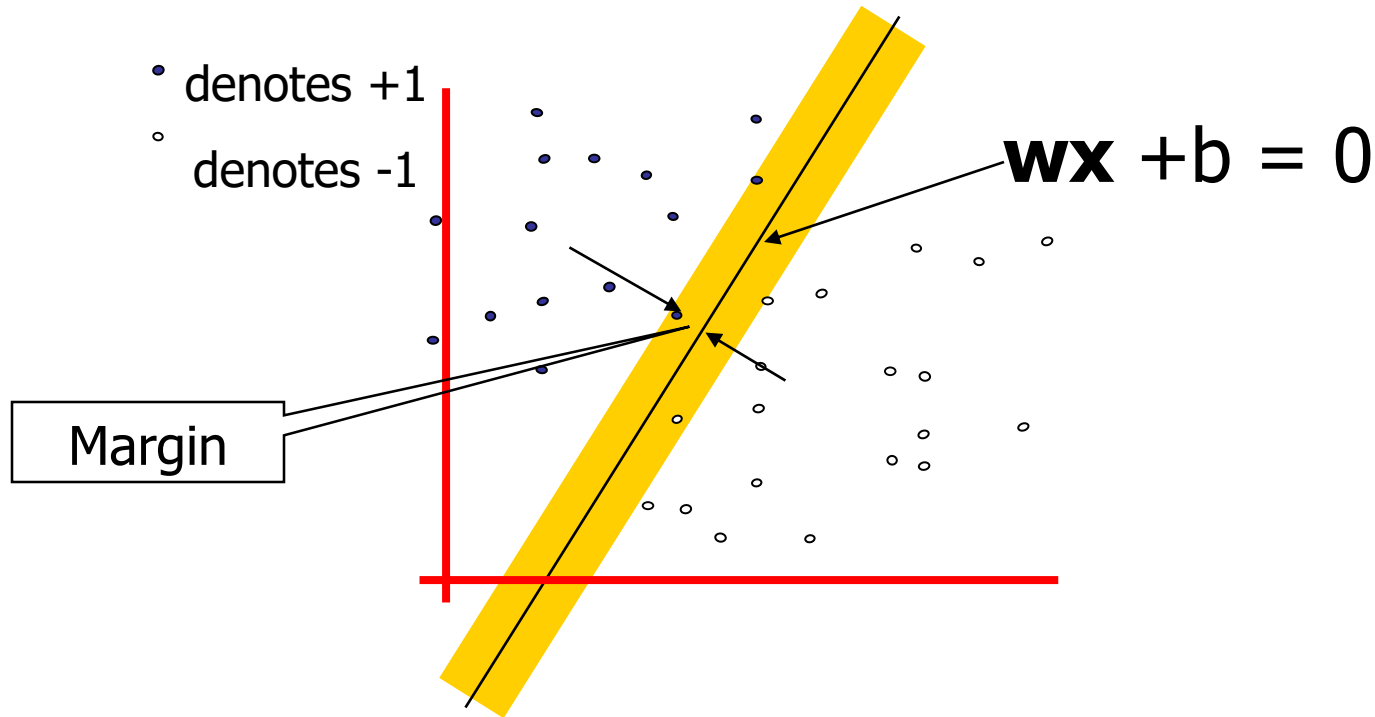
Estimate the Margin



- What is the distance expression for a point $\mathbf{x}$ to a line $\mathbf{w}\mathbf{x} + b = 0$?

$$d(\mathbf{x}) = \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\|\mathbf{w}\|_2^2}} = \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

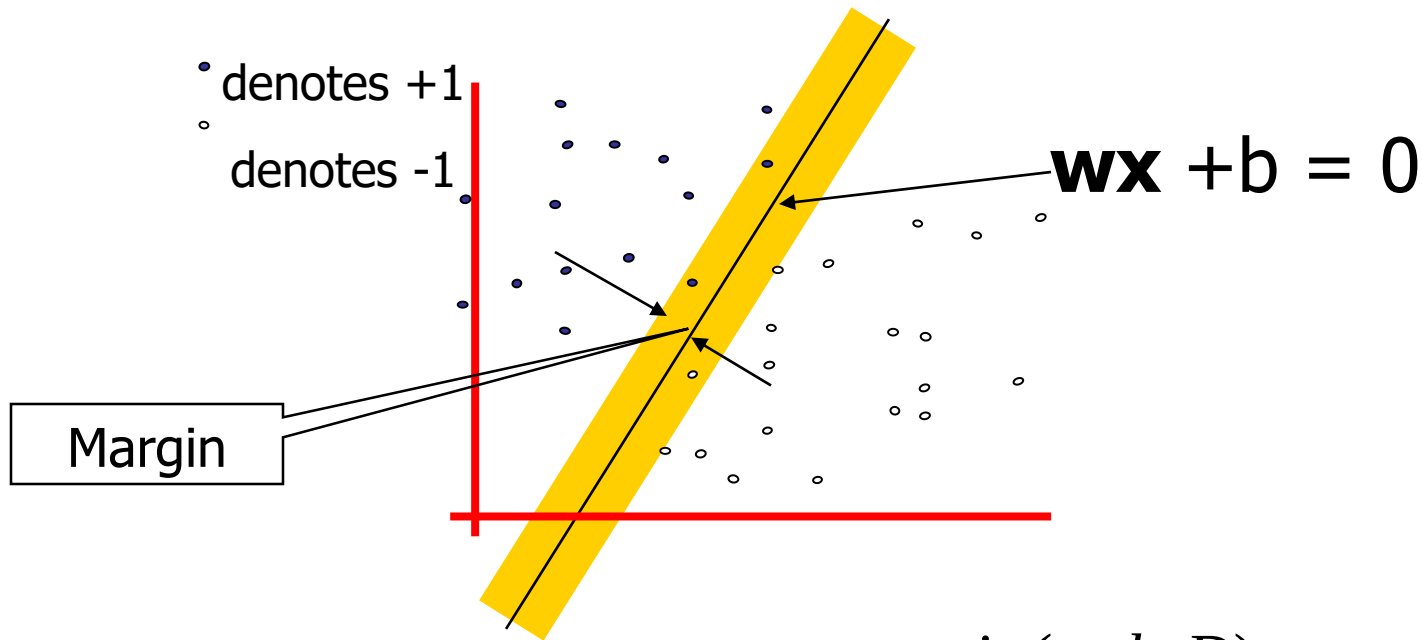
Estimate the Margin



- What is the expression for margin, given $\mathbf{w}$ and b ?

$$\text{margin} \equiv \arg \min_{\mathbf{x} \in D} d(\mathbf{x}) = \arg \min_{\mathbf{x} \in D} \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

Maximize Margin

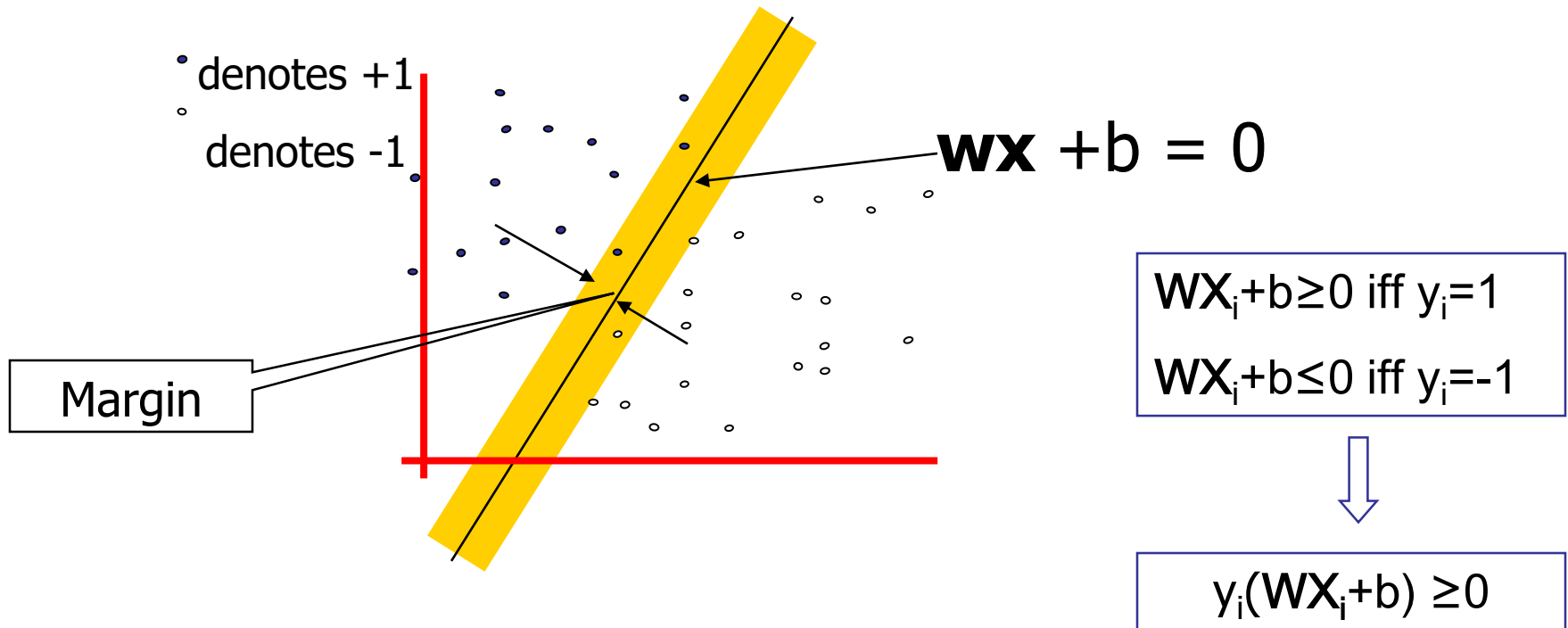


$$\operatorname{argmax}_{\mathbf{w}, b} \operatorname{margin}(\mathbf{w}, b, D)$$

$$= \operatorname{argmax}_{\mathbf{w}, b} \operatorname{argmin}_{\mathbf{x}_i \in D} d(\mathbf{x}_i)$$

$$= \operatorname{argmax}_{\mathbf{w}, b} \operatorname{argmin}_{\mathbf{x}_i \in D} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

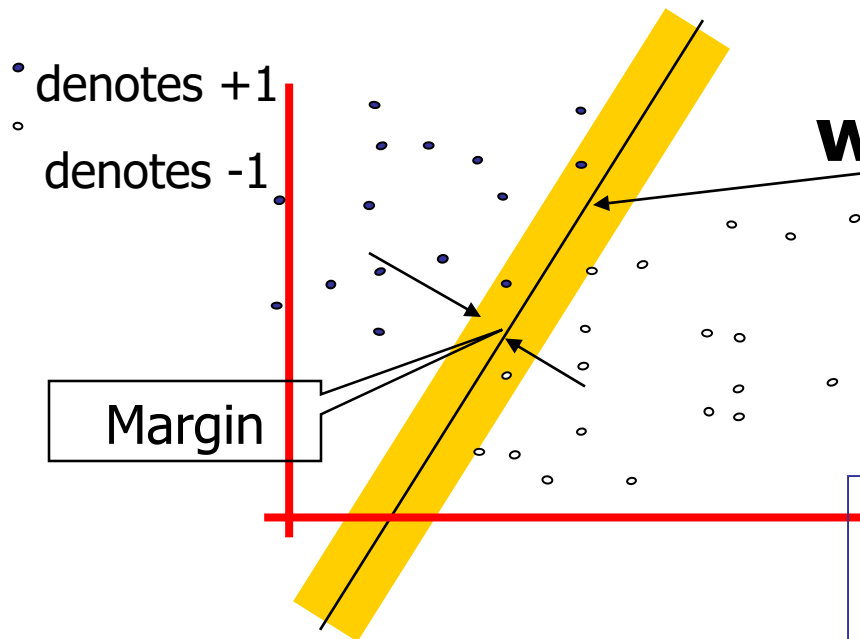
Maximize Margin



$$\operatorname{argmax}_{\mathbf{w}, b} \operatorname{argmin}_{\mathbf{x}_i \in D} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

$$\text{subject to } \forall \mathbf{x}_i \in D : y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 0$$

Maximize Margin



$$\mathbf{w}\mathbf{x}_i + b \geq 0 \text{ iff } y_i = 1$$

$$\mathbf{w}\mathbf{x}_i + b \leq 0 \text{ iff } y_i = -1$$



$$y_i(\mathbf{w}\mathbf{x}_i + b) \geq 0$$

Strategy:

$$\forall \mathbf{x}_i \in D: |b + \mathbf{x}_i \cdot \mathbf{w}| \geq 1$$

$$\mathbf{w}\mathbf{x} + b = 0$$

$$\alpha(\mathbf{w}\mathbf{x} + b) = 0 \text{ where } \alpha \neq 0$$

$$\begin{aligned} & \underset{\mathbf{w}, b}{\operatorname{argmax}} \underset{\mathbf{x}_i \in D}{\operatorname{argmin}} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}} \\ & \text{subject to } \forall \mathbf{x}_i \in D: y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 0 \end{aligned}$$



$$\begin{aligned} & \underset{\mathbf{w}, b}{\operatorname{argmin}} \sum_{i=1}^d w_i^2 \\ & \text{subject to } \forall \mathbf{x}_i \in D: y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 1 \end{aligned}$$

Maximize Margin

- How does it come ?

$$\begin{aligned} & \underset{\mathbf{w}, b}{\operatorname{argmax}} \underset{\mathbf{x}_i \in D}{\operatorname{argmin}} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}} \\ & \text{subject to } \forall \mathbf{x}_i \in D : y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 0 \\ & \forall \mathbf{x}_i \in D : |b + \mathbf{x}_i \cdot \mathbf{w}| \geq 1 \end{aligned}$$

$$\begin{aligned} & \underset{\mathbf{w}, b}{\operatorname{argmin}} \sum_{i=1}^d w_i^2 \\ & \text{subject to } \forall \mathbf{x}_i \in D : y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 1 \end{aligned}$$

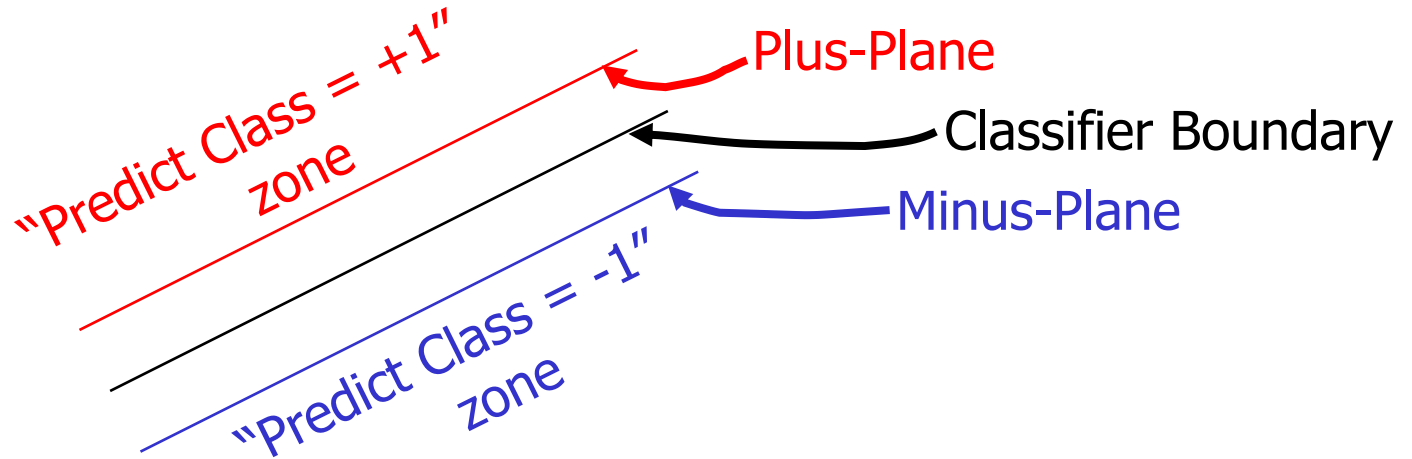
We have

$$\underset{\mathbf{w}, b}{\operatorname{argmin}} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}} = \underset{\mathbf{w}, b}{\operatorname{argmin}} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}| \times K}{\sqrt{\sum_{i=1}^d w_i^2 \times K}} = \underset{\mathbf{w}, b}{\operatorname{argmin}} \frac{1}{\sqrt{\sum_{i=1}^d w_i^2}}$$

Thus,

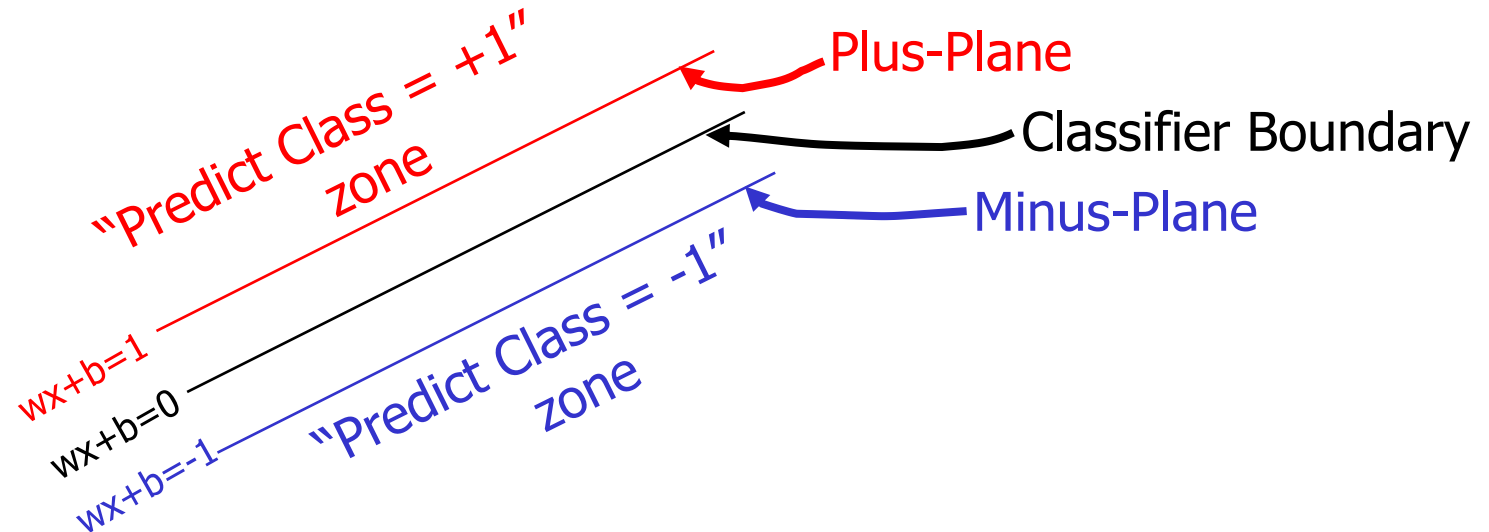
$$\underset{\mathbf{w}, b}{\operatorname{argmax}} \underset{\mathbf{x}_i \in D}{\operatorname{argmin}} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}} = \underset{\mathbf{w}, b}{\operatorname{argmax}} \frac{1}{\sqrt{\sum_{i=1}^d w_i^2}} = \underset{\mathbf{w}, b}{\operatorname{argmin}} \sum_{i=1}^d w_i^2$$

Specifying a line and margin



- How do we represent this mathematically?
- ...in m input dimensions?

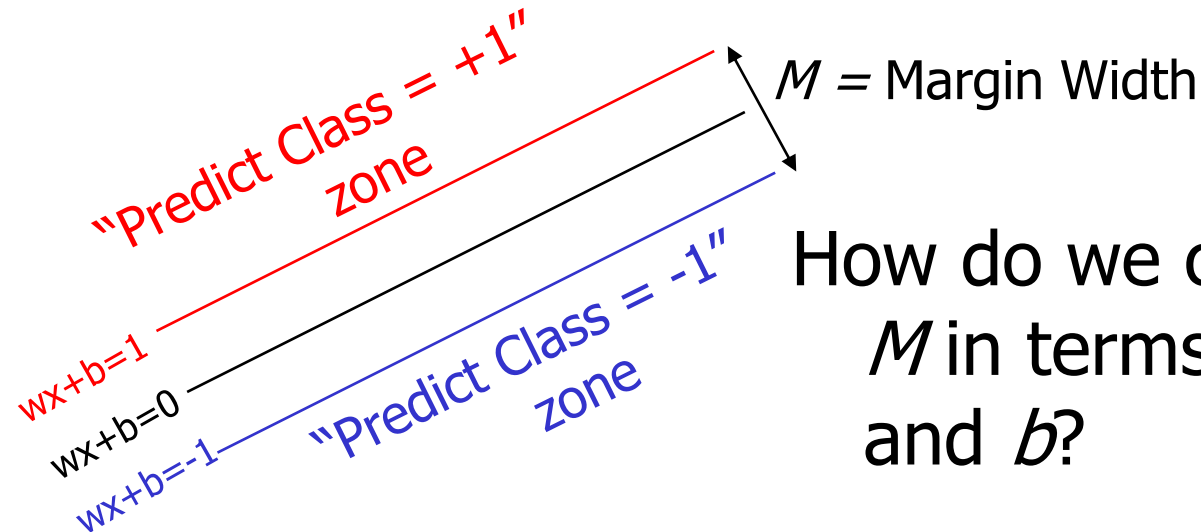
Specifying a line and margin



- Plus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = +1 \}$
- Minus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = -1 \}$

Classify as..	+1	if	$\mathbf{w} \cdot \mathbf{x} + b \geq 1$
	-1	if	$\mathbf{w} \cdot \mathbf{x} + b \leq -1$
	Universe explodes	if	$-1 < \mathbf{w} \cdot \mathbf{x} + b < 1$

Computing the margin width

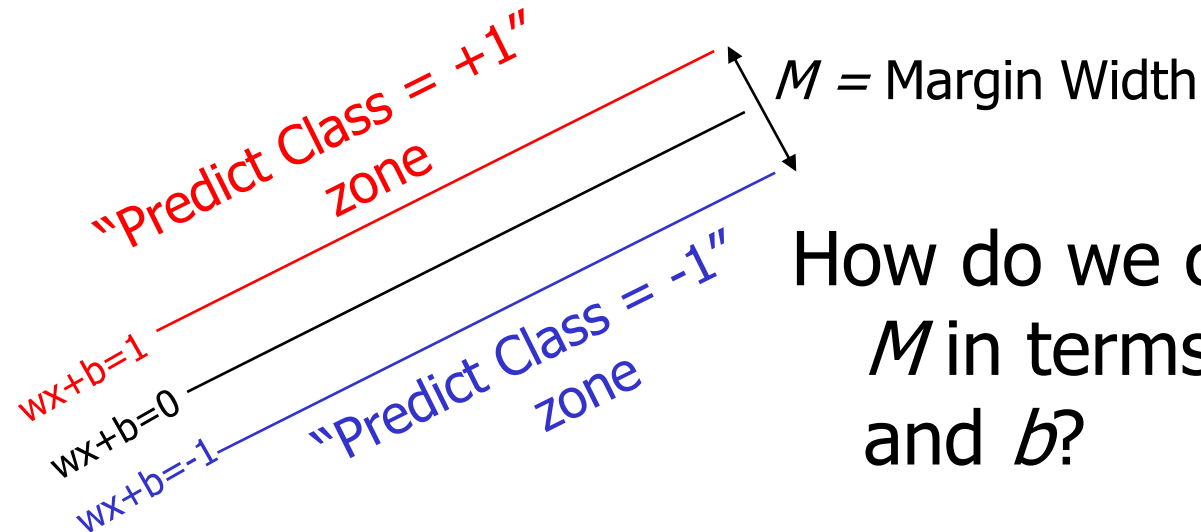


How do we compute M in terms of $\mathbf{w}$ and b ?

- Plus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = +1 \}$
- Minus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = -1 \}$

Claim: The vector $\mathbf{w}$ is perpendicular to the Plus Plane. **Why?**

Computing the margin width



How do we compute M in terms of $\mathbf{w}$ and b ?

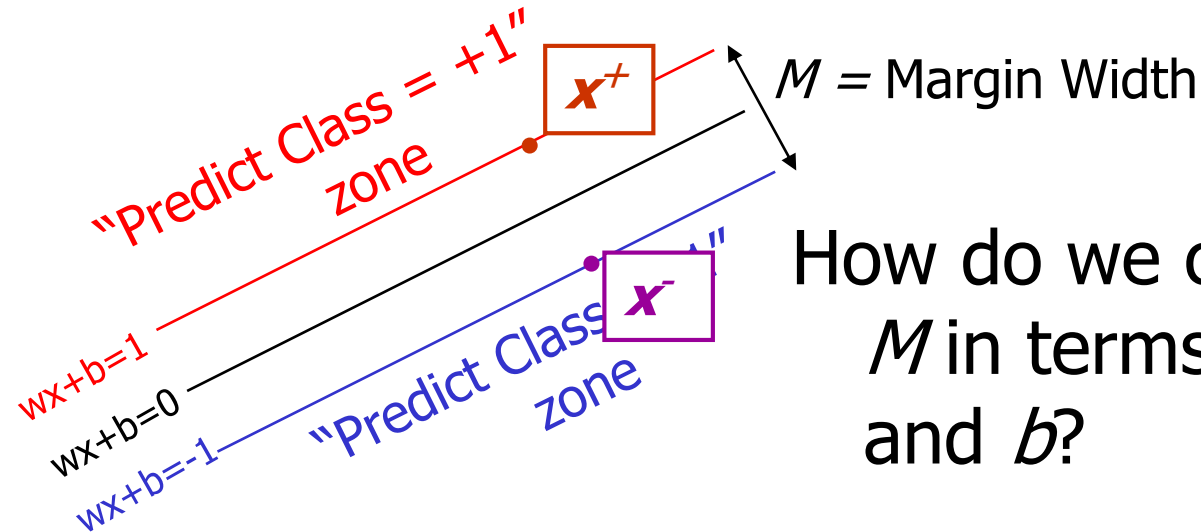
- Plus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = +1 \}$
- Minus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = -1 \}$

Claim: The vector $\mathbf{w}$ is perpendicular to the Plus Plane. **Why?**

Let $\mathbf{u}$ and $\mathbf{v}$ be two vectors on the Plus Plane. What is $\mathbf{w} \cdot (\mathbf{u} - \mathbf{v})$?

And so of course the vector $\mathbf{w}$ is also perpendicular to the Minus Plane

Computing the margin width

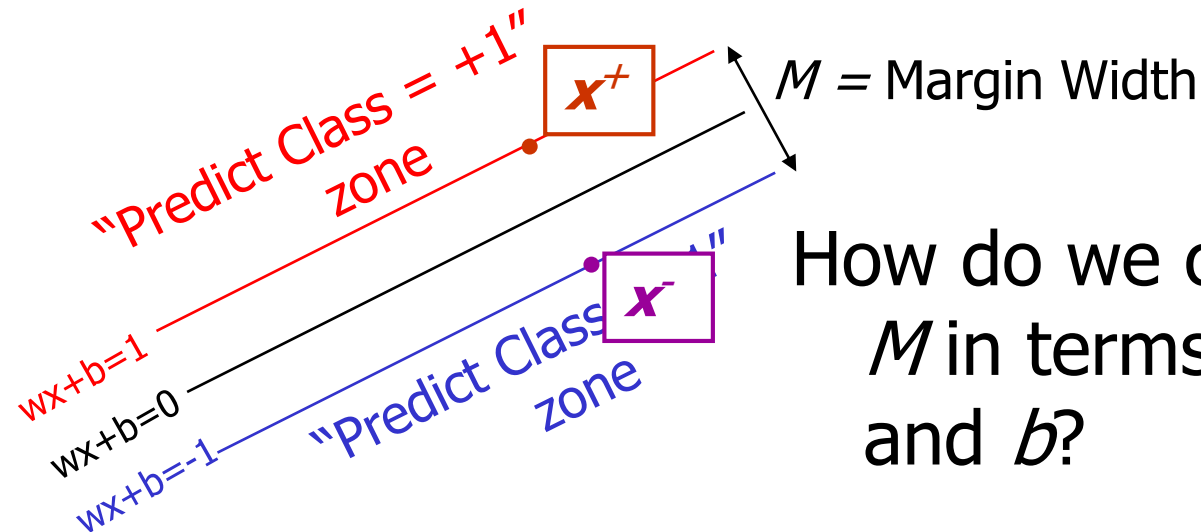


How do we compute M in terms of $\mathbf{w}$ and b ?

- Plus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = +1 \}$
- Minus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = -1 \}$
- The vector $\mathbf{w}$ is perpendicular to the Plus Plane
- Let $\mathbf{x}^-$ be any point on the minus plane
- Let $\mathbf{x}^+$ be the closest plus-plane-point to $\mathbf{x}^-$.

Any location in $\mathbb{R}^m$: not necessarily a datapoint

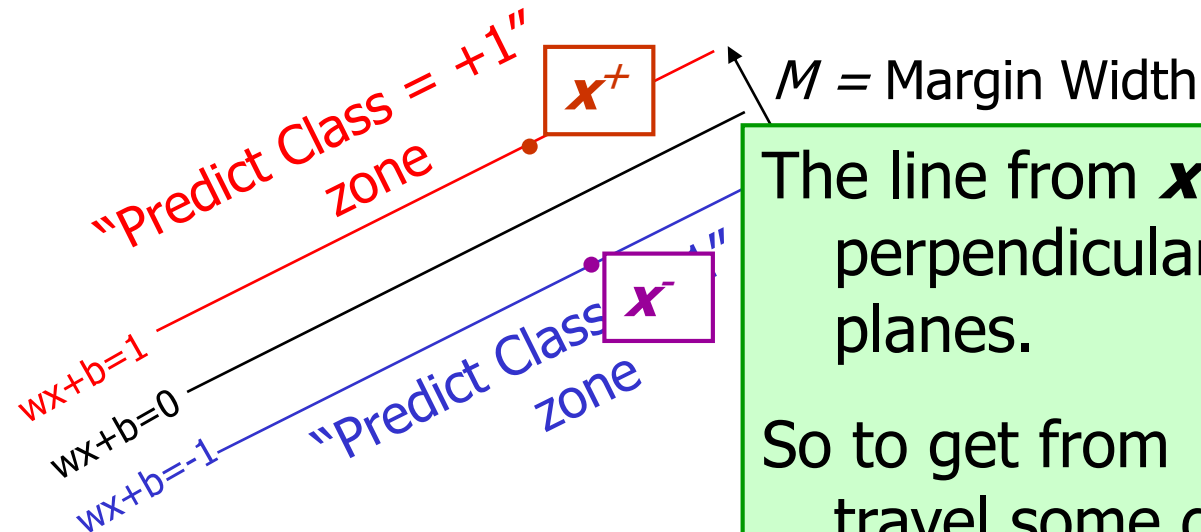
Computing the margin width



How do we compute M in terms of $\mathbf{w}$ and b ?

- Plus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = +1 \}$
- Minus-plane = $\{ \mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = -1 \}$
- The vector $\mathbf{w}$ is perpendicular to the Plus Plane
- Let $\mathbf{x}^-$ be any point on the minus plane
- Let $\mathbf{x}^+$ be the closest plus-plane-point to $\mathbf{x}^-$.
- **Claim:** $\mathbf{x}^+ = \mathbf{x}^- + \lambda \mathbf{w}$ for some value of λ . **Why?**

Computing the margin width

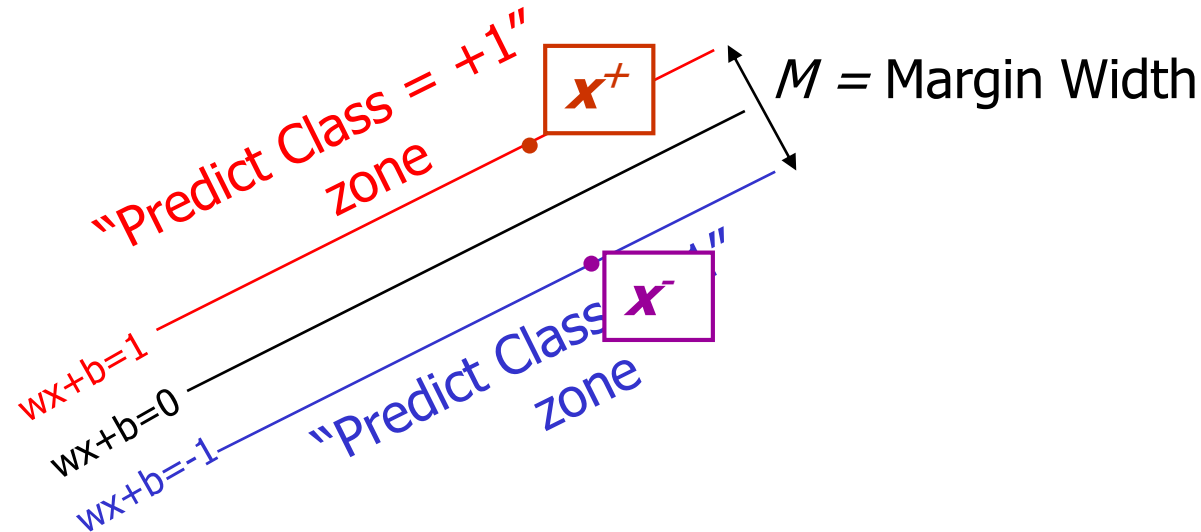


The line from x^- to x^+ is perpendicular to the planes.

So to get from x^- to x^+ travel some distance in direction w .

- Plus-plane = $\{x : w \cdot x + b = 1\}$
- Minus-plane = $\{x : w \cdot x + b = -1\}$
- The vector w is perpendicular to the Plus Plane
- Let x^- be any point on the minus plane
- Let x^+ be the closest plus-plane-point to x^- .
- **Claim:** $x^+ = x^- + \lambda w$ for some value of λ . **Why?**

Computing the margin width



What we know:

- $\mathbf{w} \cdot \mathbf{x}^+ + b = +1$
- $\mathbf{w} \cdot \mathbf{x}^- + b = -1$
- $\mathbf{x}^+ = \mathbf{x}^- + \lambda \mathbf{w}$
- $|\mathbf{x}^+ - \mathbf{x}^-| = M$

It's now easy to get M
in terms of $\mathbf{w}$ and b

Maximize Margin

- How does it come ?

$$\begin{aligned} & \underset{\mathbf{w}, b}{\operatorname{argmax}} \underset{\mathbf{x}_i \in D}{\operatorname{argmin}} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}} \\ & \text{subject to } \forall \mathbf{x}_i \in D : y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 0 \\ & \forall \mathbf{x}_i \in D : |b + \mathbf{x}_i \cdot \mathbf{w}| \geq 1 \end{aligned}$$

$$\begin{aligned} & \underset{\mathbf{w}, b}{\operatorname{argmin}} \sum_{i=1}^d w_i^2 \\ & \text{subject to } \forall \mathbf{x}_i \in D : y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 1 \end{aligned}$$

We have

$$\operatorname{argmin} \frac{|b + x_i \cdot w|}{\sqrt{\sum_{i=1}^d w_i^2}} = \operatorname{argmin} \frac{|b + x_i \cdot w| \times K}{\sqrt{\sum_{i=1}^d w_i^2 \times K}} = \frac{1}{\sqrt{\sum_{i=1}^d w_i'^2}}$$

Thus,

$$\operatorname{argmax} \operatorname{argmin} \frac{|b + x_i \cdot w|}{\sqrt{\sum_{i=1}^d w_i^2}} = \operatorname{argmax} \frac{1}{\sqrt{\sum_{i=1}^d w_i'^2}} = \operatorname{argmin} \sum_{i=1}^d w_i'^2$$

Maximum Margin Linear Classifier

$$\{\vec{w}^*, b^*\} = \operatorname{argmax}_{\vec{w}, b} \sum_{k=1}^d w_k^2$$

subject to

$$y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1$$

$$y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1$$

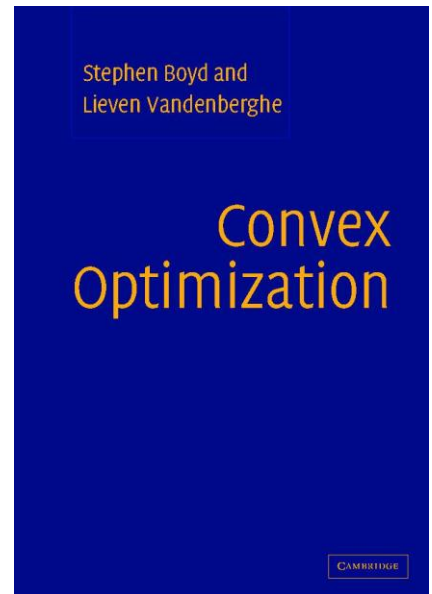
....

$$y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1$$

- How to solve it?

Learning via Quadratic Programming

- QP is a well-studied class of optimization algorithms to maximize a quadratic function of some real-valued variables subject to linear constraints.
- Detail solution of Quadratic Programming
 - [Convex Optimization Stephen P. Boyd](#)
 - Online Edition, Free for Downloading



www.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf

Quadratic Programming

Find $\arg \max_{\mathbf{u}} \quad c + \mathbf{d}^T \mathbf{u} + \frac{\mathbf{u}^T R \mathbf{u}}{2}$  Quadratic criterion

Subject to

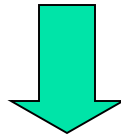
$$\begin{aligned} a_{11}u_1 + a_{12}u_2 + \dots + a_{1m}u_m &\leq b_1 \\ a_{21}u_1 + a_{22}u_2 + \dots + a_{2m}u_m &\leq b_2 \\ &\vdots \\ a_{n1}u_1 + a_{n2}u_2 + \dots + a_{nm}u_m &\leq b_n \end{aligned}$$


n additional linear inequality constraints

Quadratic Programming for the Linear Classifier

$$\{\vec{w}^*, b^*\} = \min_{\vec{w}, b} \sum_i w_i^2$$

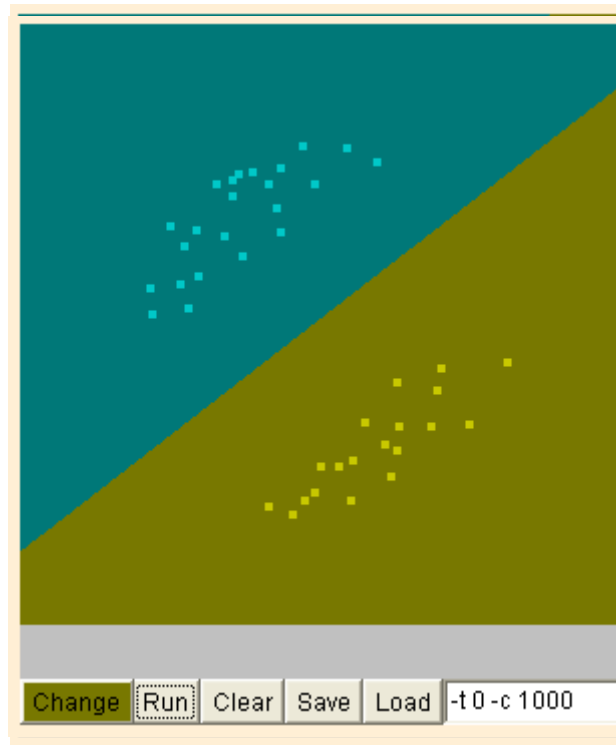
subject to $y_i (\vec{w} \cdot \vec{x}_i + b) \geq 1$ for all training data $(\vec{x}_i, y_i)$



$$\{\vec{w}^*, b^*\} = \operatorname{argmax}_{\vec{w}, b} \left\{ 0 + \vec{0} \cdot \vec{w} - \vec{w}^T \mathbf{I}_n \vec{w} \right\}$$

$$\left. \begin{array}{l} y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1 \\ y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1 \\ \dots \\ y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1 \end{array} \right\} \text{inequality constraints}$$

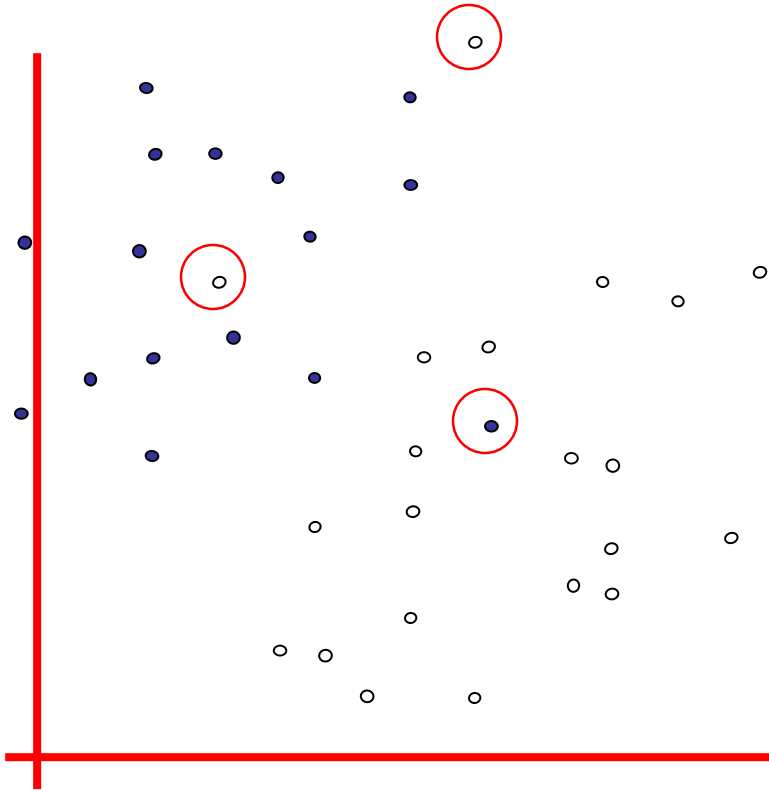
- Popular Tools - LibSVM



Uh-oh!

This is going to be a problem!
What should we do?

- denotes +1
- denotes -1

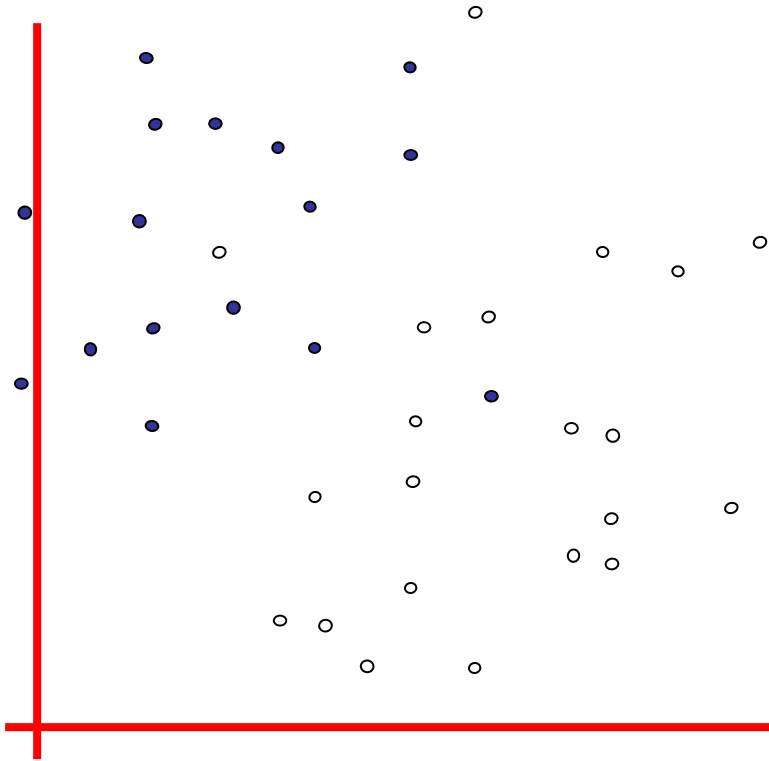


Uh-oh!

This is going to be a problem!

What should we do?

- denotes +1
- denotes -1



Idea 1:

Find minimum $\mathbf{w} \cdot \mathbf{w}$, while minimizing number of training set errors.

Problem: Two things to minimize makes for an ill-defined optimization

Uh-oh!

This is going to be a problem!

What should we do?

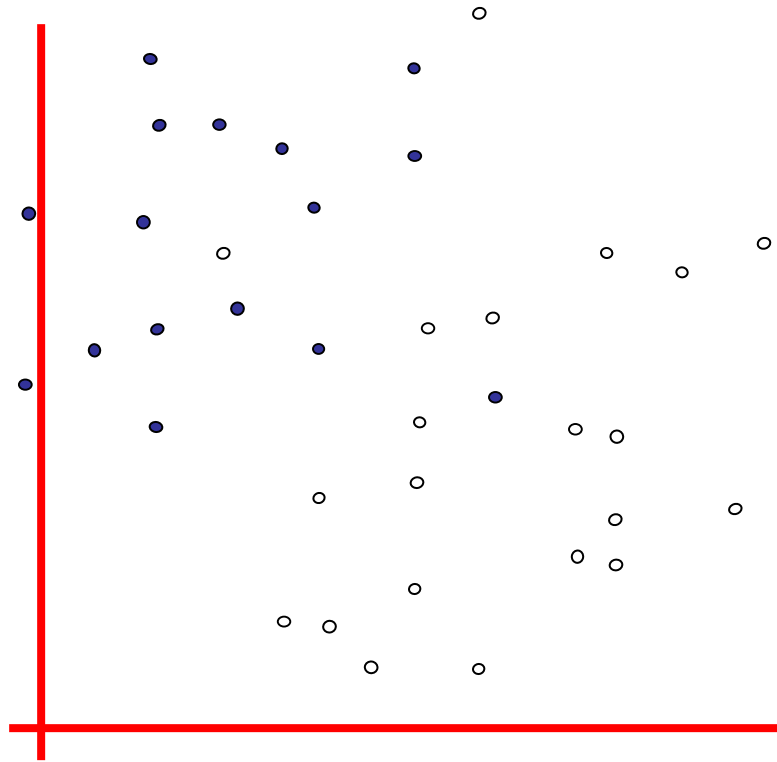
Idea 1.1:

Minimize

$\mathbf{w} \cdot \mathbf{w} + C (\#train\ errors)$

Tradeoff parameter

- denotes +1
- denotes -1



There's a serious practical problem that's about to make us reject this approach. Can you guess what it is?

Uh-oh!

This is going to be a problem!

What should we do?

- denotes +1
- denotes -1

Idea 1.1:

Minimize

$\mathbf{w} \cdot \mathbf{w} + C (\#train\ errors)$

Tradeoff parameter

Can't be expressed as a Quadratic Programming problem.

Solving it may be too slow.

(Also, doesn't distinguish between disastrous errors and near misses)

So... any other ideas?

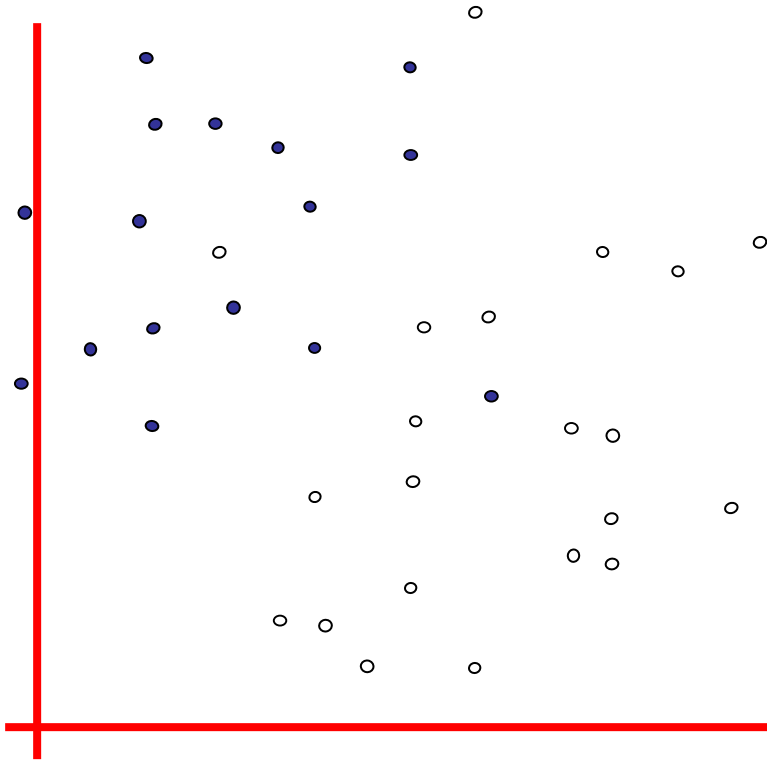
you guess what?

Uh-oh!

This is going to be a problem!

What should we do?

- denotes +1
- denotes -1

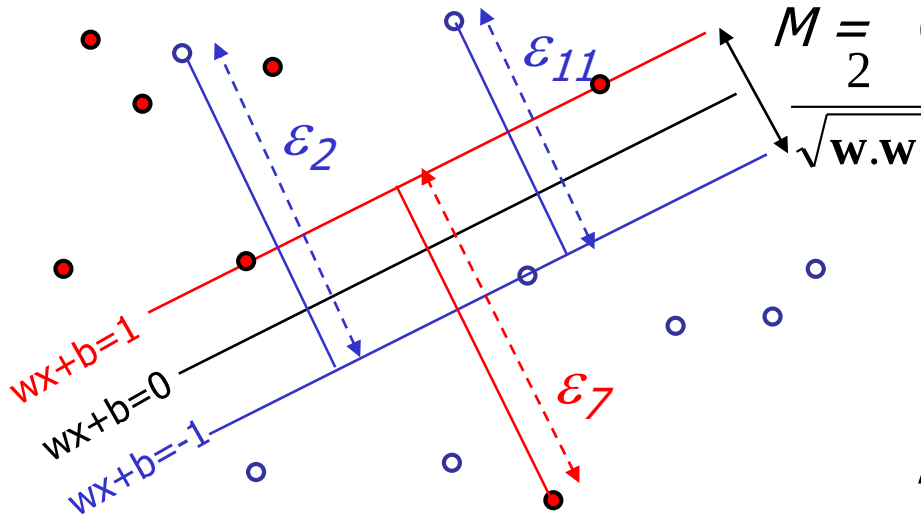


Idea 2.0:

Minimize

$W.W + C$ (*distance of error points to their correct place*)

Learning Maximum Margin with Noise



Given guess of $\mathbf{w}$, b we can

- Compute sum of distances of points to their correct zones

- Compute the margin width

Assume R datapoints, each $(\mathbf{x}_k, y_k)$ where $y_k = \pm 1$

What should our quadratic optimization criterion be?

Minimize
$$\frac{1}{2} \mathbf{w} \cdot \mathbf{w} + C \sum_{k=1}^R \varepsilon_k$$

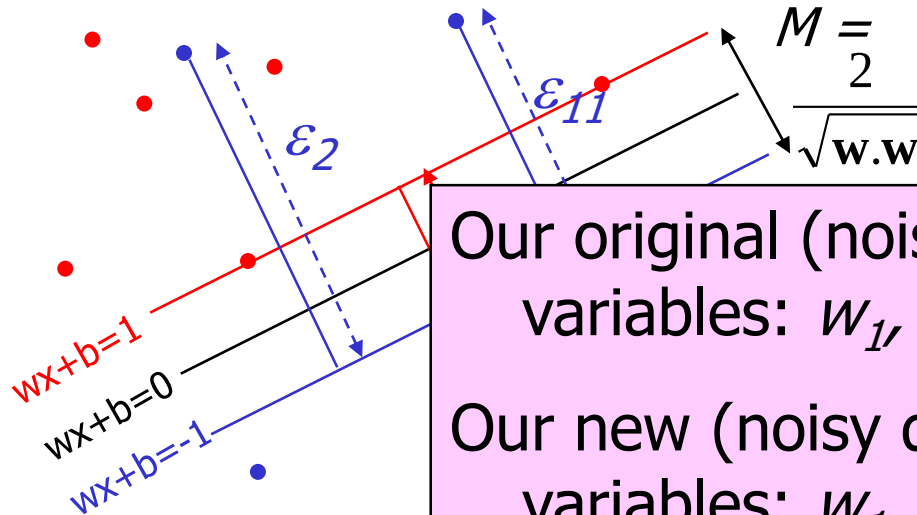
How many constraints will we have? R

What should they be?

$\mathbf{w} \cdot \mathbf{x}_k + b \geq 1 - \varepsilon_k$ if $y_k = 1$

$\mathbf{w} \cdot \mathbf{x}_k + b \leq -1 + \varepsilon_k$ if $y_k = -1$

Learning Maximum Margin with Noise



$m = \#$ input dimensions

Given $\mathbf{g}$ we can

- Compute sum of distances

Our original (noiseless data) QP had $m+1$ variables: $w_1, w_2, \dots, w_m$ and b .

Our new (noisy data) QP has $m+1+R$ variables: $w_1, w_2, \dots, w_m, b, \epsilon_k, \epsilon_1, \dots, \epsilon_R$

What should our quadratic optimization criterion be?

Minimize

$$\frac{1}{2} \mathbf{w} \cdot \mathbf{w} + C \sum_{k=1}^R \epsilon_k$$

How many constraints do we have? R

$R = \#$ records

What should they be?

$$\mathbf{w} \cdot \mathbf{x}_k + b \geq 1 - \epsilon_k \text{ if } y_k = 1$$

$$\mathbf{w} \cdot \mathbf{x}_k + b \leq -1 + \epsilon_k \text{ if } y_k = -1$$

Support Vector Machine (SVM) for Noisy Data

$$\{\vec{w}^*, b^*\} = \min_{\vec{w}, b} \sum_{i=1}^d w_i^2 + c \sum_{j=1}^N \varepsilon_j$$

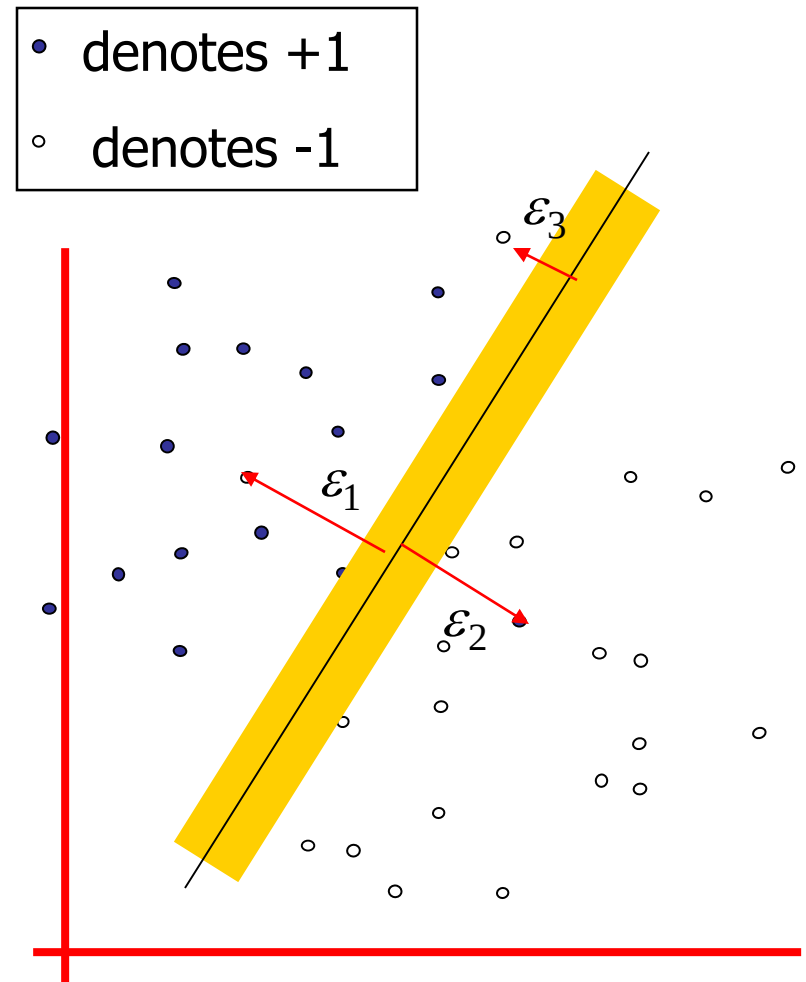
$$y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1 - \varepsilon_1$$

$$y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1 - \varepsilon_2$$

...

$$y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1 - \varepsilon_N$$

- Any problem with the above formulism?



Support Vector Machine (SVM) for Noisy Data

$$\{\vec{w}^*, b^*\} = \min_{\vec{w}, b} \sum_{i=1}^d w_i^2 + c \sum_{j=1}^N \varepsilon_j$$

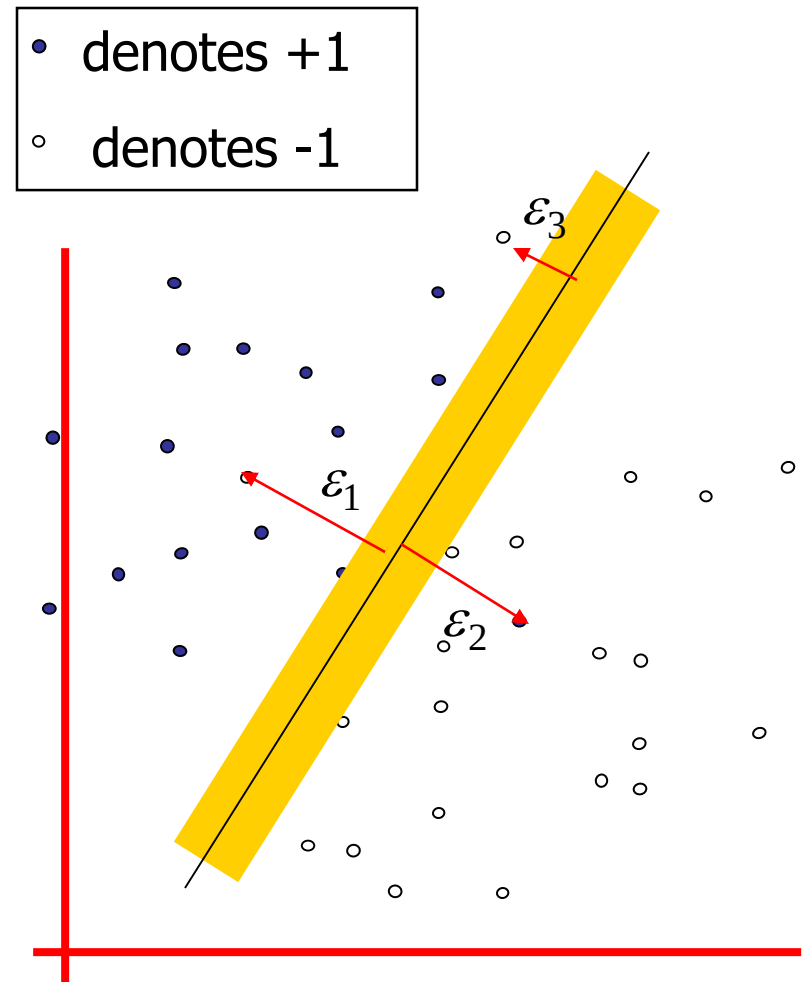
$$y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1 - \varepsilon_1, \varepsilon_1 \geq 0$$

$$y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1 - \varepsilon_2, \varepsilon_2 \geq 0$$

...

$$y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1 - \varepsilon_N, \varepsilon_N \geq 0$$

- Balance the trade off between margin and classification errors



Support Vector Machine for Noisy Data

$$\{\vec{w}^*, b^*\} = \operatorname{argmin}_{\vec{w}, b} \sum_i w_i^2 + c \sum_{j=1}^N \varepsilon_j$$

$$\left. \begin{array}{l} y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1 - \varepsilon_1, \varepsilon_1 \geq 0 \\ y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1 - \varepsilon_2, \varepsilon_2 \geq 0 \\ \dots \\ y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1 - \varepsilon_N, \varepsilon_N \geq 0 \end{array} \right\} \text{inequality constraints}$$

How do we determine the appropriate value for c ?

Therefore, the problem of maximizing the margin is equivalent to

$$\begin{array}{ll} \text{Minimize} & J(w) = \frac{1}{2} \|w\|^2 \\ \text{Subject to} & y_i(w^T x_i + b) \geq 1 \quad \forall i \end{array}$$

- Notice that $J(w)$ is a quadratic function, which means that there exists a single global minimum and no local minima

To solve this problem, we will use classical Lagrangian optimization techniques

- We first present the Kuhn-Tucker Theorem, which provides an essential result for the interpretation of Support Vector Machines

(Kuhn-Tucker Theorem)

Given an optimization problem with convex domain $\Omega \subseteq R^N$

$$\begin{array}{lll} \text{Minimize} & f(z) & z \in \Omega \\ \text{Subject to} & g_i(z) \leq 0 & i = 1..k \\ & h_i(z) = 0 & i = 1..m \end{array}$$

- with $f \in C^1$ convex and g_i, h_i affine, necessary & sufficient conditions for a normal point z^* to be an optimum are the existence of α^*, β^* such that

$$\partial L(z^*, \alpha^*, \beta^*) / \partial z = 0$$

$$\partial L(z^*, \alpha^*, \beta^*) / \partial \beta = 0$$

$$\alpha_i^* g_i(z^*) = 0 \quad i = 1..k \quad \text{where:} \quad L(z, \alpha, \beta) = f(z) + \sum_{i=1}^k \alpha_i g_i(z) + \sum_{i=1}^m \beta_i h_i(z)$$

$$g_i(z^*) \leq 0 \quad i = 1..k$$

$$\alpha_i^* \geq 0 \quad i = 1..k$$

- $L(z, \alpha, \beta)$ is known as a *generalized Lagrangian function*
- The third condition is known as the Karush-Kuhn-Tucker (KKT) complementary condition
 - It implies that for active constraints $\alpha_i \geq 0$; and for inactive constraints $\alpha_i = 0$
 - As we will see in a minute, the KKT condition allows us to identify the training examples that define the largest margin hyperplane. These examples will be known as Support Vectors

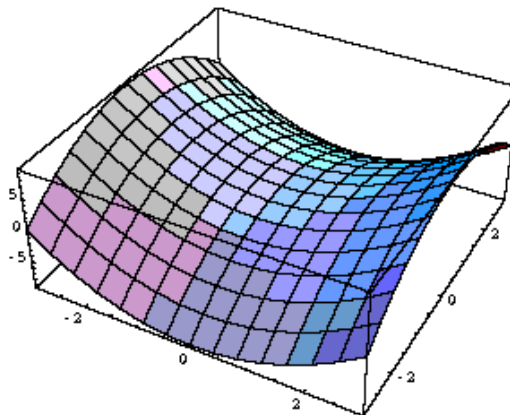
[Cristianini and Shawe-Taylor, 2000]

The Lagrangian dual problem

Constrained minimization of $J(w) = 1/2\|w\|^2$ is solved by introducing the Lagrangian

$$L_P(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i [y_i(w^T x_i + b) - 1]$$

- which yields an unconstrained optimization problem that is solved by:
 - minimizing L_P w.r.t. the primal variables w and b , and
 - maximizing L_P w.r.t. the dual variables $\alpha_i \geq 0$ (the Lagrange multipliers)
- Thus, the optimum is defined by a saddle point
- This is known as the Lagrangian primal problem



A saddle point

Solution

- To simplify the primal problem, we eliminate the primal variables (w, b) using the first Kuhn-Tucker condition $\partial J / \partial z = 0$

- Differentiating $L_P(w, b, \alpha)$ with respect to w and b , and setting to zero yields

$$\partial L_P(w, b, \alpha) / \partial w = 0 \quad \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i$$

$$\partial L_P(w, b, \alpha) / \partial b = 0 \quad \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0$$

- Expansion of L_P yields

$$L_P(w, b, \alpha) = \frac{1}{2} w^T w - \sum_{i=1}^N \alpha_i y_i w^T x_i - b \sum_{i=1}^N \alpha_i y_i + \sum_{i=1}^N \alpha_i$$

- Using the optimality condition $\partial J / \partial w = 0$, the first term in L_P can be expressed as

$$\begin{aligned} w^T w &= w^T \sum_{i=1}^N \alpha_i y_i x_i = \sum_{i=1}^N \alpha_i y_i w^T x_i = \\ &= \sum_{i=1}^N \alpha_i y_i \left(\sum_{j=1}^N \alpha_j y_j x_j \right)^T x_i = \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j \end{aligned}$$

- The second term in L_P can be expressed in the same way
- The third term in L_P is zero by virtue of the optimality condition $\partial J / \partial b = 0$

[Haykin, 1999]

- Merging these expressions together we obtain

$$L_D(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j x_i^T x_j$$

- Subject to the (simpler) constraints $\alpha_i \geq 0$ and $\sum_{i=1}^N \alpha_i y_i = 0$
- This is known as the **Lagrangian dual problem**

Comments

- We have transformed the problem of finding a saddle point for $L_P(w, b)$ into the easier one of maximizing $L_D(\alpha)$
 - Notice that $L_D(\alpha)$ depends on the Lagrange multipliers α , not on (w, b)
- The primal problem scales with dimensionality (w has one coefficient for each dimension), whereas the dual problem scales with the amount of training data (there is one Lagrange multiplier per example)
- Moreover, in $L_D(\alpha)$ training data appears only as dot products $x_i^T x_j$
 - As we will see in the next lecture, this property can be cleverly exploited to perform the classification in a higher (e.g., infinite) dimensional space

Support Vectors

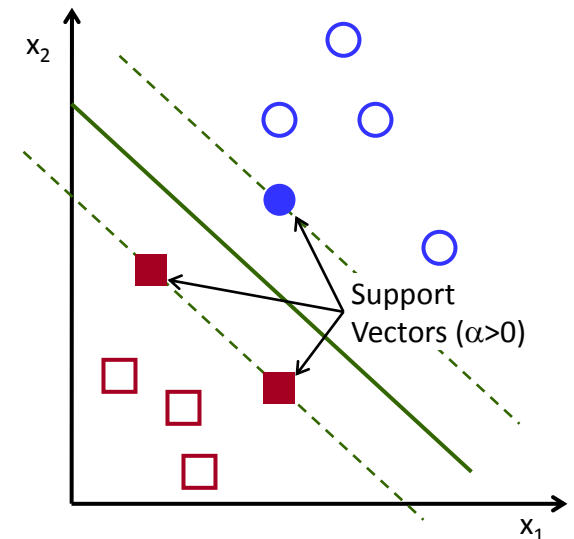
The KKT complementary condition states that, for every point in the training set, the following equality must hold

$$\alpha_i [y_i(w^T x_i + b) - 1] = 0 \quad \forall i = 1..N$$

- Therefore, $\forall x$, either $\alpha_i = 0$ or $y_i(w^T x_i + b - 1) = 0$ must hold
 - Those points for which $\alpha_i > 0$ must then lie on one of the two hyperplanes that define the largest margin (the term $y_i(w^T x_i + b - 1)$ becomes zero only at these hyperplanes)
 - These points are known as the **Support Vectors**
 - All the other points must have $\alpha_i = 0$
- Note that only the SVs contribute to defining the optimal hyperplane

$$\frac{\partial J(w, b, \alpha)}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i$$

- NOTE: the bias term b is found from the KKT complementary condition on the support vectors
- Therefore, the complete dataset could be replaced by only the support vectors, and the separating hyperplane would be the same



The Dual Form of QP

$$\text{Maximize } \sum_{k=1}^R \alpha_k - \frac{1}{2} \sum_{k=1}^R \sum_{l=1}^R \alpha_k \alpha_l Q_{kl} \text{ where } Q_{kl} = y_k y_l (\mathbf{x}_k \cdot \mathbf{x}_l)$$

Subject to these constraints:

$$0 \leq \alpha_k \leq C \quad \forall k \quad \sum_{k=1}^R \alpha_k y_k = 0$$

Then define:

$$\mathbf{w} = \sum_{k=1}^R \alpha_k y_k \mathbf{x}_k$$

An Equivalent QP

$$\text{Maximize } \sum_{k=1}^R \alpha_k - \frac{1}{2} \sum_{k=1}^R \sum_{l=1}^R \alpha_k \alpha_l Q_{kl} \text{ where } Q_{kl} = y_k y_l (\mathbf{x}_k \cdot \mathbf{x}_l)$$

Subject to these constraints:

$$0 \leq \alpha_k \leq C \quad \forall k \quad \sum_{k=1}^R \alpha_k y_k = 0$$

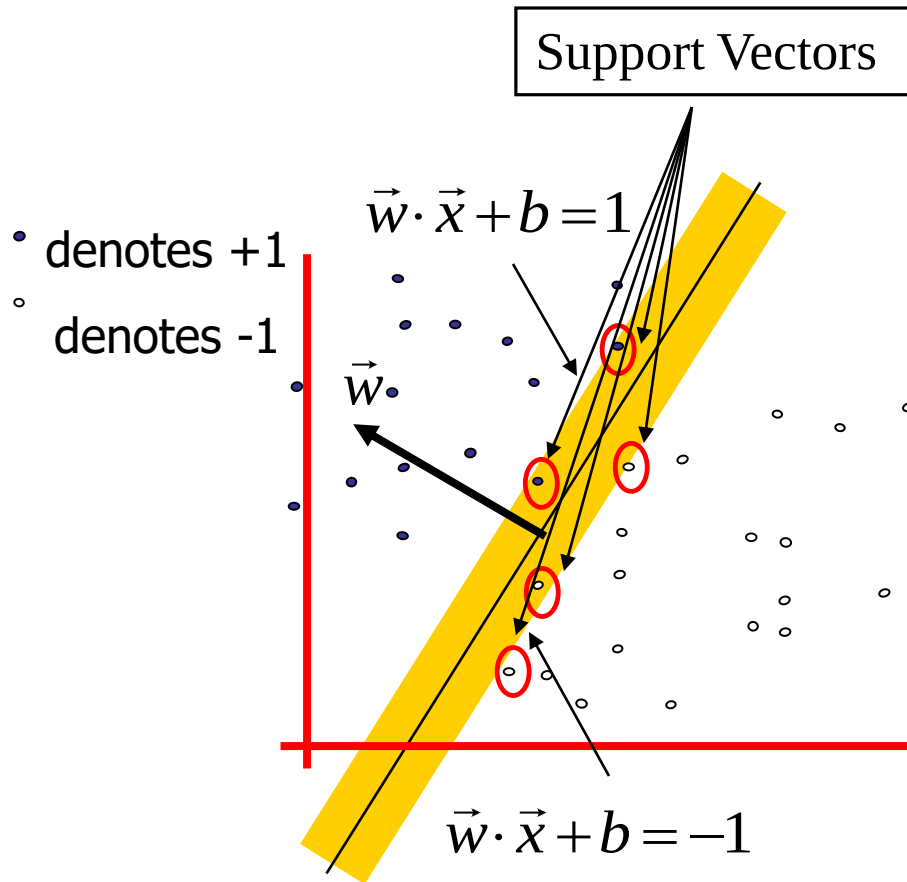
Then define:

$$\mathbf{w} = \sum_{k=1}^R \alpha_k y_k \mathbf{x}_k$$

Datapoints with $\alpha_k > 0$ will be the support vectors

..so this sum only needs to be over the support vectors.

Support Vectors



$$\forall i : \alpha_i (y_i (\vec{w} \cdot \vec{x}_i + b) - (1 - \varepsilon_i)) = 0$$

$\alpha_i = 0$ for non-support vectors
 $\alpha_i \neq 0$ for support vectors

$$\mathbf{w} = \sum_{k=1}^R \alpha_k y_k \mathbf{x}_k$$

Decision boundary is
determined only by those
support vectors !

The Dual Form of QP

$$\text{Maximize } \sum_{k=1}^R \alpha_k - \frac{1}{2} \sum_{k=1}^R \sum_{l=1}^R \alpha_k \alpha_l Q_{kl} \text{ where } Q_{kl} = y_k y_l (\mathbf{x}_k \cdot \mathbf{x}_l)$$

Subject to these constraints:

$$0 \leq \alpha_k \leq C \quad \forall k$$

$$\sum_{k=1}^R \alpha_k y_k = 0$$

Then define:

$$\mathbf{w} = \sum_{k=1}^R \alpha_k y_k \mathbf{x}_k$$

Then classify with:

$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

How to determine b ?

An Equivalent QP: Determine b

$$\{\vec{w}^*, b^*\} = \operatorname{argmin}_{\vec{w}, b} \sum_i w_i^2 + c \sum_{j=1}^N \varepsilon_j$$

$$y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1 - \varepsilon_1, \varepsilon_1 \geq 0$$

$$y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1 - \varepsilon_2, \varepsilon_2 \geq 0$$

....

$$y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1 - \varepsilon_N, \varepsilon_N \geq 0$$



$$b^* = \operatorname{argmin}_{b, \{\varepsilon_i\}_{i=1}^N} \sum_{j=1}^N \varepsilon_j$$

$$y_1 (\vec{w} \cdot \vec{x}_1 + b) \geq 1 - \varepsilon_1, \varepsilon_1 \geq 0$$

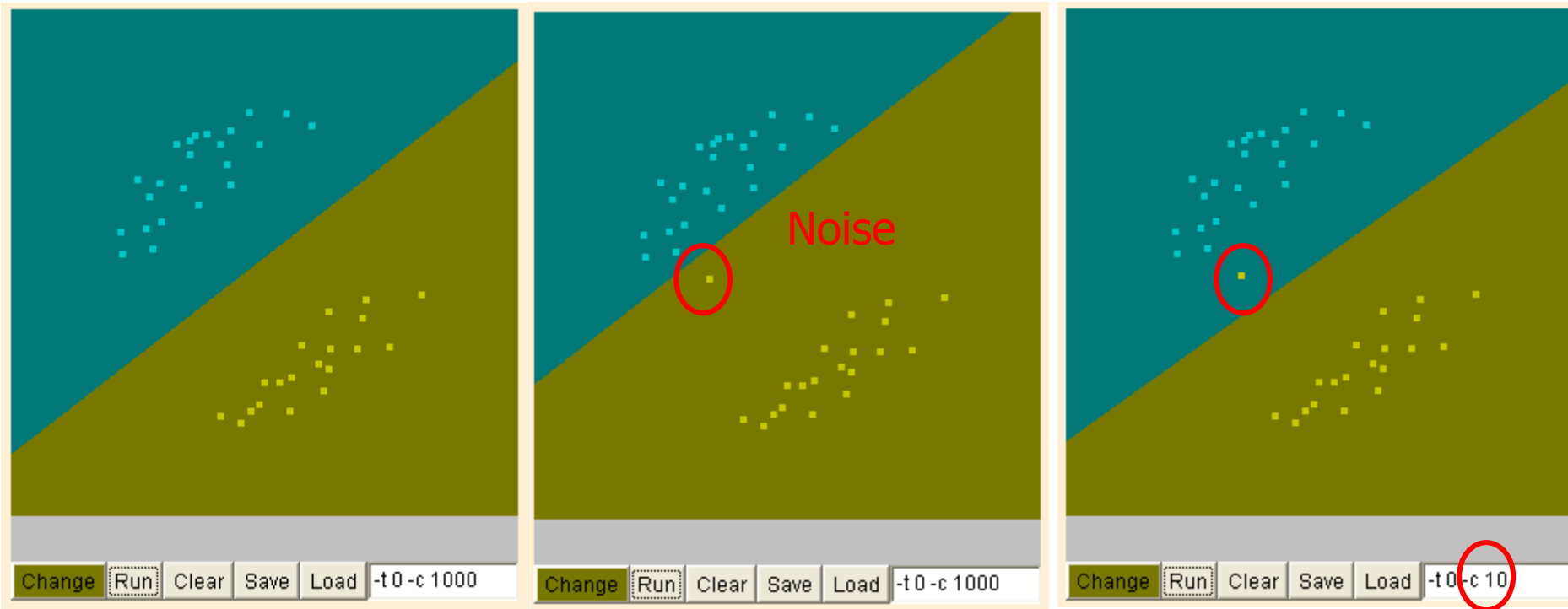
$$y_2 (\vec{w} \cdot \vec{x}_2 + b) \geq 1 - \varepsilon_2, \varepsilon_2 \geq 0$$

....

$$y_N (\vec{w} \cdot \vec{x}_N + b) \geq 1 - \varepsilon_N, \varepsilon_N \geq 0$$

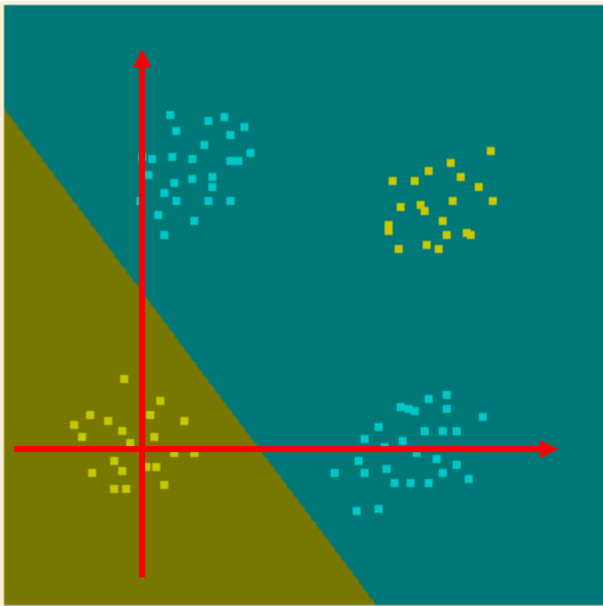
A linear programming problem !

- Parameter **c** is used to control the fitness



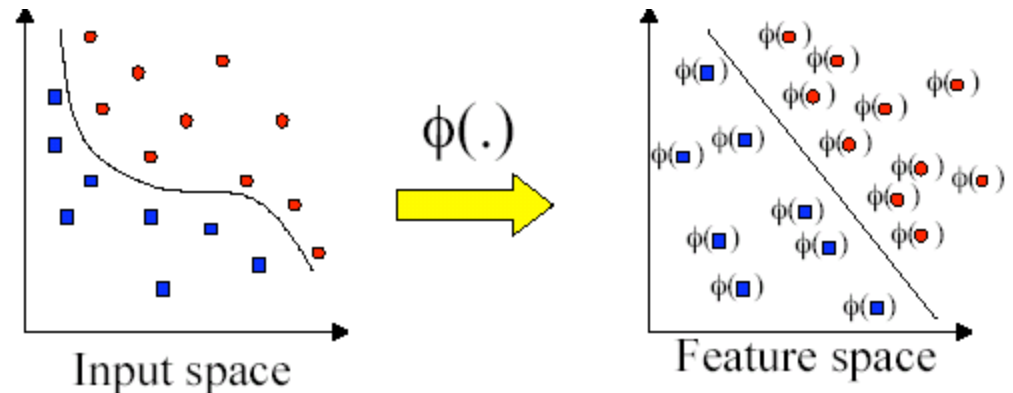
Feature Transformation ?

- The problem is non-linear
- Find some trick to transform the input
- Linear separable after **Feature Transformation**
- What **Features** should we use ?



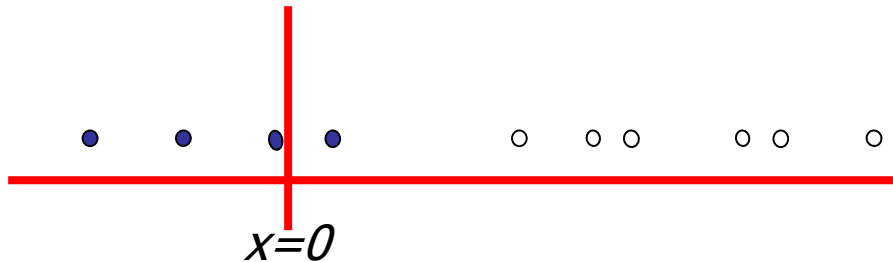
XOR Problem

Basic Idea :



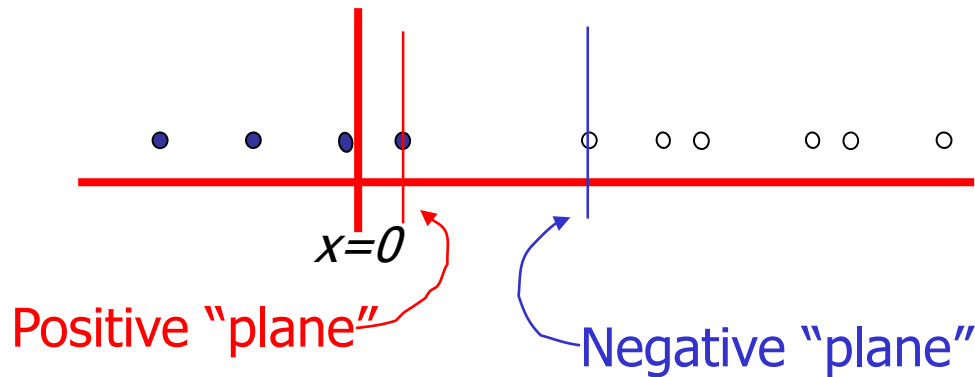
Suppose we're in 1-dimension

What would
SVMs do with
this data?



Suppose we're in 1-dimension

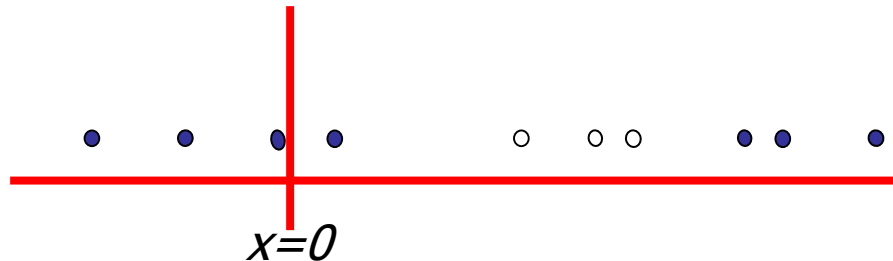
Not a big surprise



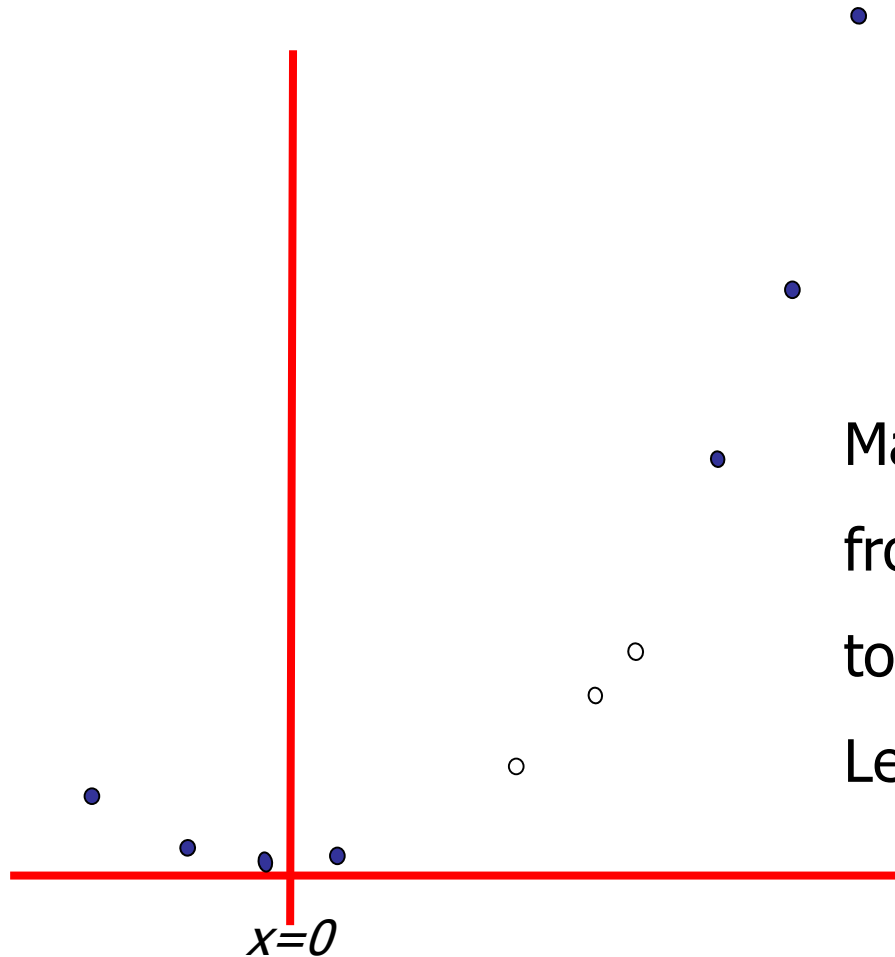
Harder 1-dimensional dataset

That's wiped the smirk off SVM's face.

What can be done about this?



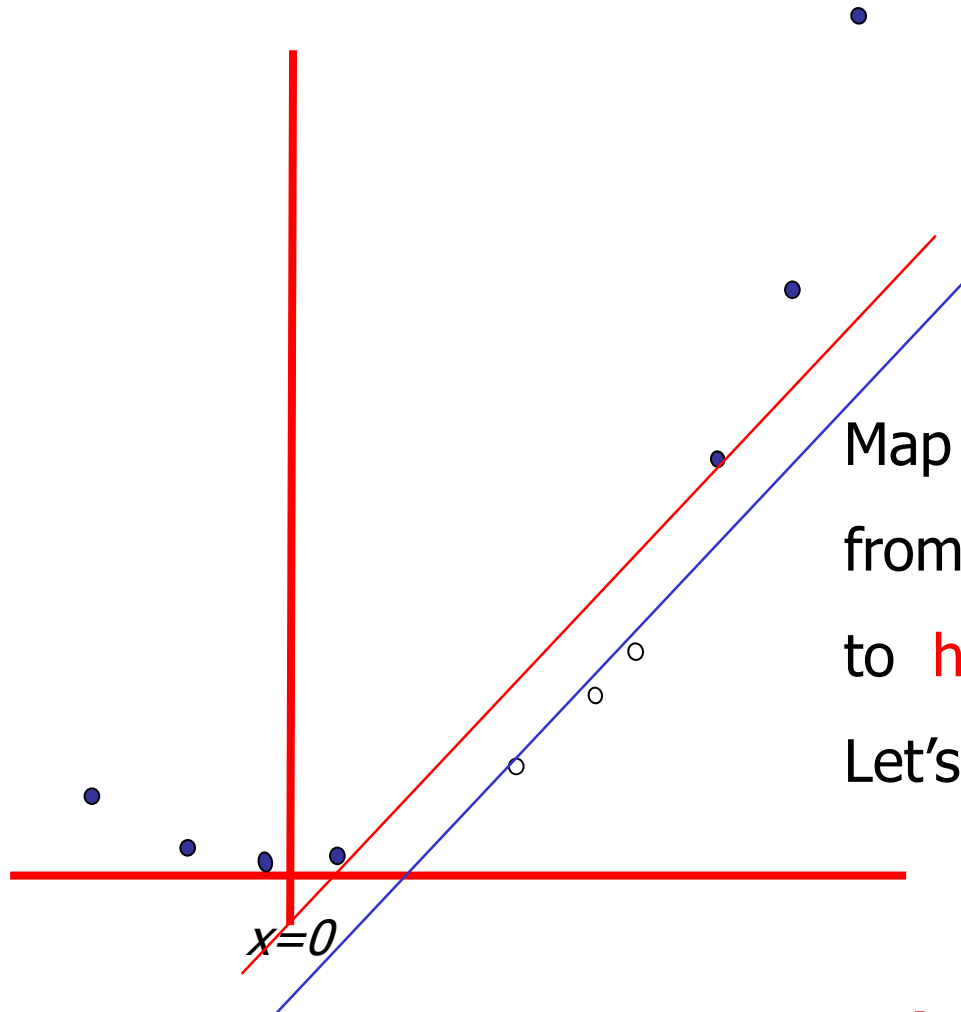
Harder 1-dimensional dataset



Map the data
from **low-dimensional space**
to **high-dimensional space**
Let's permit them here too

$$\mathbf{z}_k = (x_k, x_k^2)$$

Harder 1-dimensional dataset



Map the data
from **low-dimensional space**
to **high-dimensional space**
Let's permit them here too

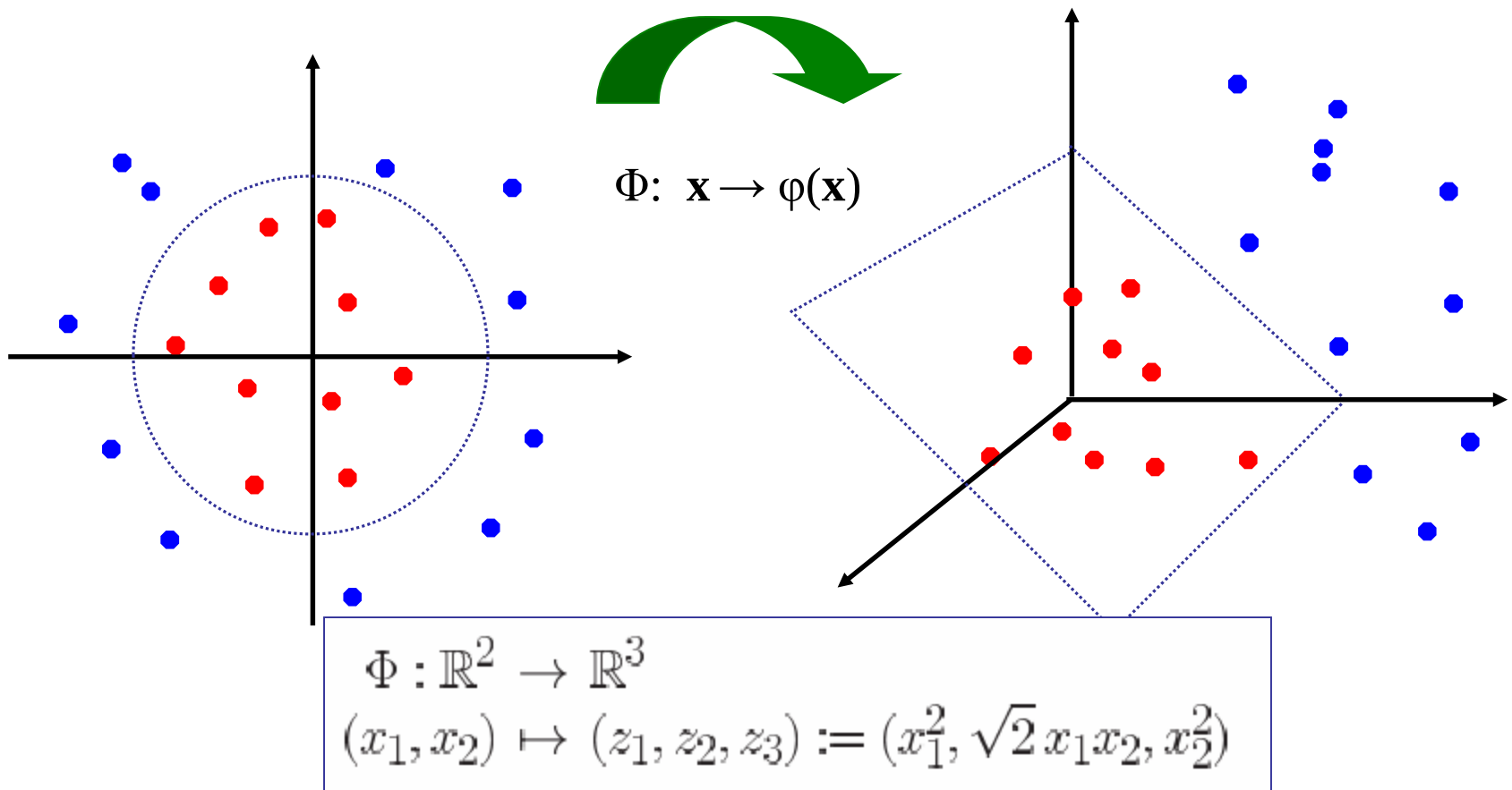
$$\mathbf{z}_k = (x_k, x_k^2)$$

Feature Enumeration

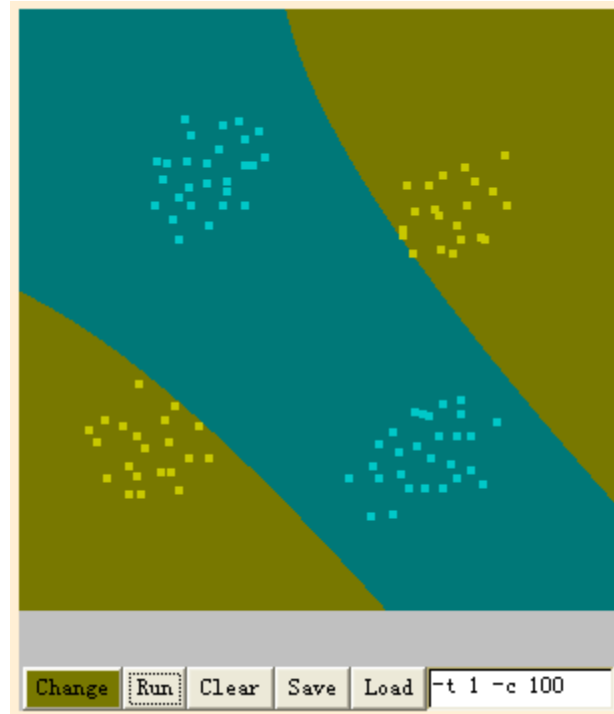
$$x_k \xrightarrow{\text{transform}} \Phi(x_k) = z_k$$

Non-linear SVMs: Feature spaces

- General idea:** the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



- Polynomial features for the XOR problem



Efficiency Problem in Computing Feature

- Feature space Mapping

Preprocess the data with

$$\begin{aligned}\Phi : \mathcal{X} &\rightarrow \mathcal{H} \\ x &\mapsto \Phi(x),\end{aligned}$$

where $\mathcal{H}$ is a dot product space, and learn the mapping from $\Phi(x)$ to y .

- Example: all 2 degree Monomials

$$\begin{aligned}\Phi : \mathbb{R}^2 &\rightarrow \mathbb{R}^3 \\ (x_1, x_2) &\mapsto (z_1, z_2, z_3) := (x_1^2, \sqrt{2} x_1 x_2, x_2^2)\end{aligned}$$

This use of kernel function to avoid carrying out $\Phi(x)$ explicitly is known as the **kernel trick**

$$\langle \Phi(x), \Phi(x') \rangle = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)(x_1'^2, \sqrt{2} x_1' x_2', x_2'^2)^\top$$

← 9 Multiplication

$$\begin{aligned}\text{kernel trick} &= \langle x, x' \rangle^2 \\ &= : k(x, x')\end{aligned}$$

← 3 Multiplication

→ the dot product in $\mathcal{H}$ can be computed in $\mathbb{R}^2$

Kernel methods

Let's now see how to put together all these concepts

- Assume that our original feature vector x lives in a space R^D
- We are interested in non-linearly projecting x onto a higher dimensional implicit space $\varphi(x) \in R^{D1}$ ($D1 > D$) where classes have a better chance of being linearly separable
 - Notice that we are not guaranteeing linear separability, we are only saying that we have a better chance because of Cover's theorem

- The separating hyperplane in R^{D1} will be defined by

$$\sum_{j=1}^{D1} w_j \varphi_j(x) + b = 0$$

- To eliminate the bias term b , let's augment the feature vector in the implicit space with a constant dimension $\varphi_0(x) = 1$
- Using vector notation, the resulting hyperplane becomes

$$w^T \varphi(x) = 0$$

- From our previous results, the optimal (maximum margin) hyperplane in the implicit space is given by

$$w = \sum_{i=1}^N \alpha_i y_i \varphi(x_i)$$

- Merging this optimal weight vector with the hyperplane equation

$$\begin{aligned}
 w^T \varphi(x) &= 0 \\
 \Rightarrow \left(\sum_{i=1}^N \alpha_i y_i \varphi(x_i) \right)^T \varphi(x) &= 0 \\
 \Rightarrow \sum_{i=1}^N \alpha_i y_i \varphi(x_i)^T \varphi(x) &= 0
 \end{aligned}$$

- and, since $\varphi^T(x_i) \varphi(x_j) = K(x_i, x_j)$, the optimal hyperplane becomes

$$\sum_{i=1}^N \alpha_i y_i K(x_i, x) = 0$$

- Therefore, classification of an unknown example x is performed by computing the weighted sum of the kernel function with respect to the support vectors x_i (remember that only the support vectors have non-zero dual variables α_i)

How do we compute dual variables α_i in the implicit space?

- Very simple: we use the same optimization problem as before, and replace the dot product $\varphi^T(x_i)\varphi(x_j)$ with the kernel $K(x_i, x_j)$
- The Lagrangian dual problem for the non-linear SVM is simply

$$L_D(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i^T, x_j)$$

- subject to the constraints

$$\begin{cases} \sum_{i=1}^N \alpha_i y_i = 0 \\ 0 \leq \alpha_i \leq C \quad i = 1 \dots N \end{cases}$$

How do we select the implicit mapping $\varphi(x)$?

- As we saw in the example a few slides back, we will normally select a kernel function first, and then determine the implicit mapping $\varphi(x)$ that it corresponds to

Then, how do we select the kernel function $K(x_i, x_j)$?

- We must select a kernel for which an implicit mapping exists, this is, a kernel that can be expressed as the dot-product of two vectors

For which kernels $K(x_i, x_j)$ does there exist an implicit mapping $\varphi(x)$?

- The answer is given by Mercer's Condition

Mercer's Condition

Let $K(x, x')$ be a continuous symmetric kernel that is defined in the closed interval $a \leq x \leq b$

- The kernel can be expanded in the series:

$$K(x, x') = \sum_{i=1}^{\infty} \lambda_i \varphi_i(x) \varphi_i(x')$$

- Strictly speaking, the space where $\varphi(x)$ resides is a Hilbert space, a “generalization” of an Euclidean space where the inner product can be any inner product, not just the scalar dot product [Burges, 1998]
- With positive coefficients $\lambda_i > 0 \quad \forall i$
- For this expansion to be valid and for it to converge absolutely and uniformly, it is necessary and sufficient that the condition

$$\int_a^b \int_a^b K(x, x') \psi(x) \psi(x') dx dx' \geq 0$$

- holds for all $\psi(\cdot)$ for which $\int_a^b \psi^2(x) dx \leq \infty$
 - The functions $\varphi_i(x)$ are called eigenfunctions of the expansion, and the numbers λ_i are the eigenvalues. The fact that all of the eigenvalues are positive means that the kernel is positive definite
- Notice that the dimensionality of the implicit space can be infinitely large
- Mercer's Condition only tells us whether a kernel is actually an inner-product kernel, but it does not tell us how to construct the functions $\varphi_i(x)$ for the expansion

[Kaykin, 1999]

Which kernels meet Mercer's condition?

- Polynomial kernels

$$K(x, x') = (x^T x' + 1)^p$$

- The degree of the polynomial is a user-specified parameter

- Radial basis function kernels

$$K(x, x') = \exp\left(-\frac{1}{2\sigma^2} \|x - x'\|^2\right)$$

- The width σ is a user-specified parameter, but the number of radial basis functions and their centers are determined automatically by the number of support vectors and their values

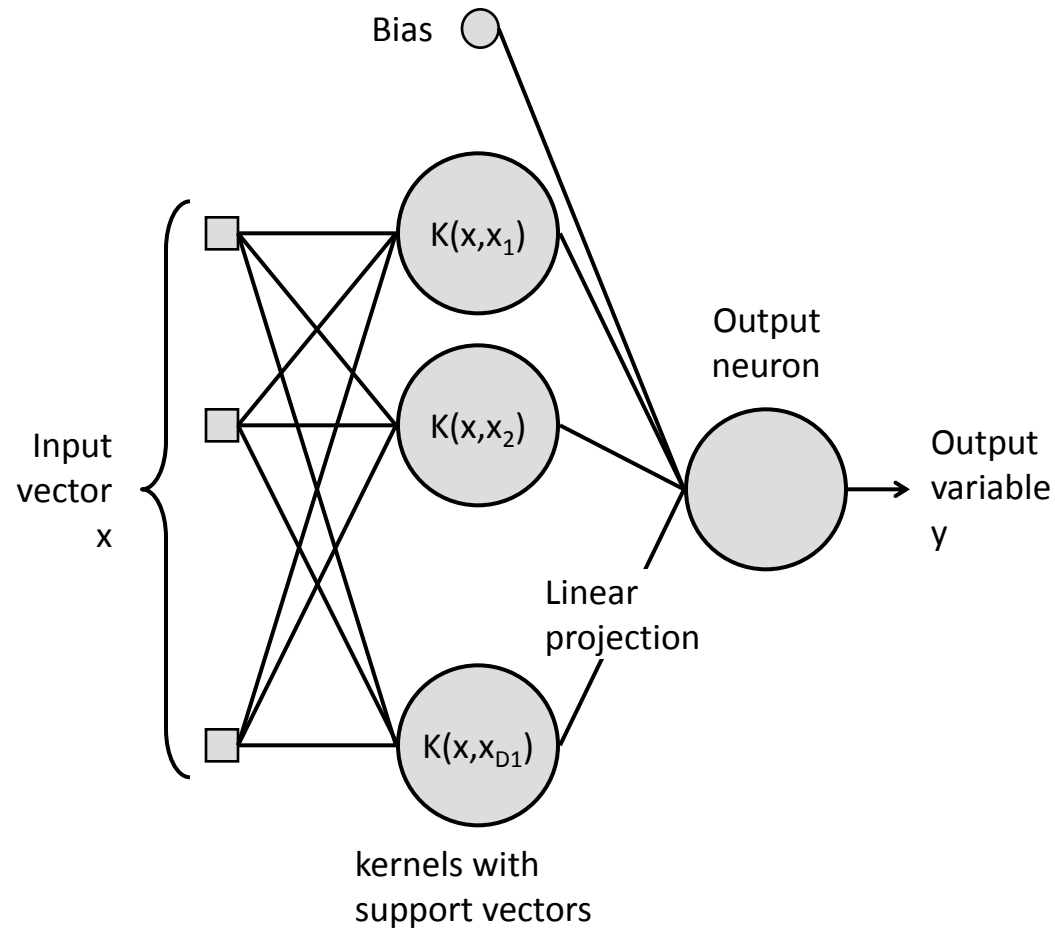
- Two-layer perceptron

$$K(x, x') = \tanh(\beta_0 x^T x' + \beta_1)$$

- The number of hidden neurons and their weight vectors are determined automatically by the number of support vectors and their values, respectively. The H-O weights are the Lagrange multipliers α_i
- However, this kernel will only meet Mercer's condition for certain values of β_0 and β_1

[Burges, 1998; Kaykin, 1999]

Architecture of an SVM



[Kaykin, 1999]

Techniques for Constructing New Kernels.

Given valid kernels $k_1(\mathbf{x}, \mathbf{x}')$ and $k_2(\mathbf{x}, \mathbf{x}')$, the following new kernels will also be valid:

$$k(\mathbf{x}, \mathbf{x}') = ck_1(\mathbf{x}, \mathbf{x}')$$

$$k(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})k_1(\mathbf{x}, \mathbf{x}')f(\mathbf{x}')$$

$$k(\mathbf{x}, \mathbf{x}') = q(k_1(\mathbf{x}, \mathbf{x}'))$$

$$k(\mathbf{x}, \mathbf{x}') = \exp(k_1(\mathbf{x}, \mathbf{x}'))$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}')$$

$$k(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}')k_2(\mathbf{x}, \mathbf{x}')$$

$$k(\mathbf{x}, \mathbf{x}') = k_3(\phi(\mathbf{x}), \phi(\mathbf{x}'))$$

$$k(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{A} \mathbf{x}'$$

$$k(\mathbf{x}, \mathbf{x}') = k_a(\mathbf{x}_a, \mathbf{x}'_a) + k_b(\mathbf{x}_b, \mathbf{x}'_b)$$

$$k(\mathbf{x}, \mathbf{x}') = k_a(\mathbf{x}_a, \mathbf{x}'_a)k_b(\mathbf{x}_b, \mathbf{x}'_b)$$

where $c > 0$ is a constant, $f(\cdot)$ is any function, $q(\cdot)$ is a polynomial with nonnegative coefficients, $\phi(\mathbf{x})$ is a function from $\mathbf{x}$ to $\mathbb{R}^M$, $k_3(\cdot, \cdot)$ is a valid kernel in $\mathbb{R}^M$, $\mathbf{A}$ is a symmetric positive semidefinite matrix, $\mathbf{x}_a$ and $\mathbf{x}_b$ are variables (not necessarily disjoint) with $\mathbf{x} = (\mathbf{x}_a, \mathbf{x}_b)$, and k_a and k_b are valid kernel functions over their respective spaces.

Numerical example

To illustrate the operation of a non-linear SVM we will solve the classical XOR problem

- Dataset
 - Class 1: $x_1 = (-1, -1)$, $x_4 = (+1, +1)$
 - Class 2: $x_2 = (-1, +1)$, $x_3 = (+1, -1)$
- Kernel function
 - Polynomial of order 2: $K(x, x') = (x^T x' + 1)^2$

Solution

- The implicit mapping can be shown to be 5-dimensional
$$\varphi(x) = [1 \quad \sqrt{2}x_{i,1} \quad \sqrt{2}x_{i,2} \quad \sqrt{2}x_{i,1}x_{i,2} \quad x_{i,1}^2 \quad x_{i,2}^2]^T$$
- To achieve linear separability, we will use $C = \infty$
- The objective function for the dual problem becomes

$$L_D(\alpha) = \alpha_1 + \alpha_2 + \alpha_3 + \alpha_4 - \frac{1}{2} \sum_{i=1}^4 \sum_{j=1}^4 \alpha_i \alpha_j y_i y_j k_{ij}$$

- subject to the constraints
$$\begin{cases} \sum_{i=1}^N \alpha_i y_i = 0 \\ 0 \leq \alpha_i \leq C \quad i = 1 \dots N \end{cases}$$

[Cherkassky and Mulier, 1998; Haykin, 1999]

- where the inner product is represented as a 4×4 K matrix

$$K = \begin{bmatrix} 9 & 1 & 1 & 1 \\ 1 & 9 & 1 & 1 \\ 1 & 1 & 9 & 1 \\ 1 & 1 & 1 & 9 \end{bmatrix}$$

- Optimizing with respect to the Lagrange multipliers leads to the following system of equations

$$9\alpha_1 - \alpha_2 - \alpha_3 + \alpha_4 = 1$$

$$-\alpha_1 + 9\alpha_2 + \alpha_3 - \alpha_4 = 1$$

$$-\alpha_1 + \alpha_2 + 9\alpha_3 - \alpha_4 = 1$$

$$\alpha_1 - \alpha_2 - \alpha_3 + 9\alpha_4 = 1$$

- whose solution is $\alpha_1 = \alpha_2 = \alpha_3 = \alpha_4 = 0.125$

- Thus, all data points are support vectors in this case

- For this simple problem, it is worthwhile to write the decision surface in terms of the polynomial expansion

$$w = \sum_{i=1}^4 \alpha_i y_i \varphi(x_i) = [0 \quad 0 \quad 0 \quad 1/\sqrt{2} \quad 0 \quad 0]^T$$

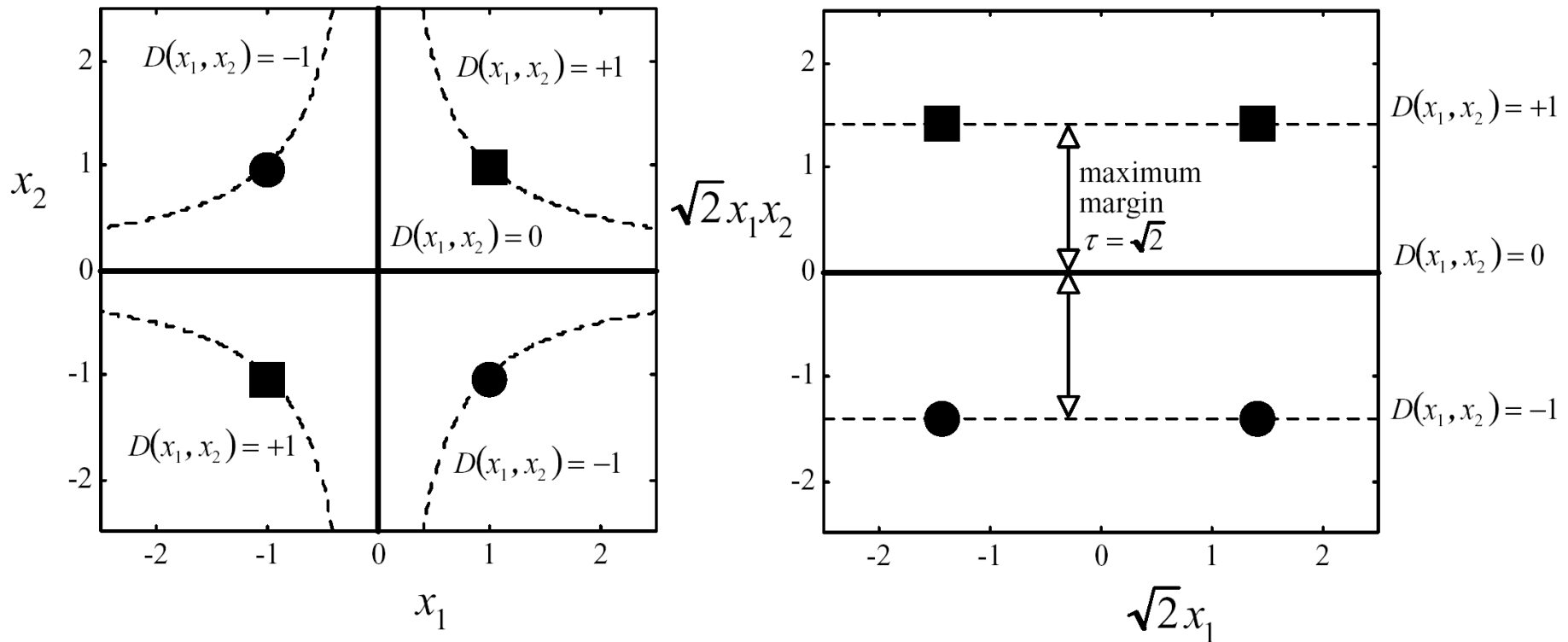
- resulting in the intuitive non-linear discriminant function

$$g(x) = \sum_{i=1}^6 w_i \varphi_i(x) = x_1 x_2$$

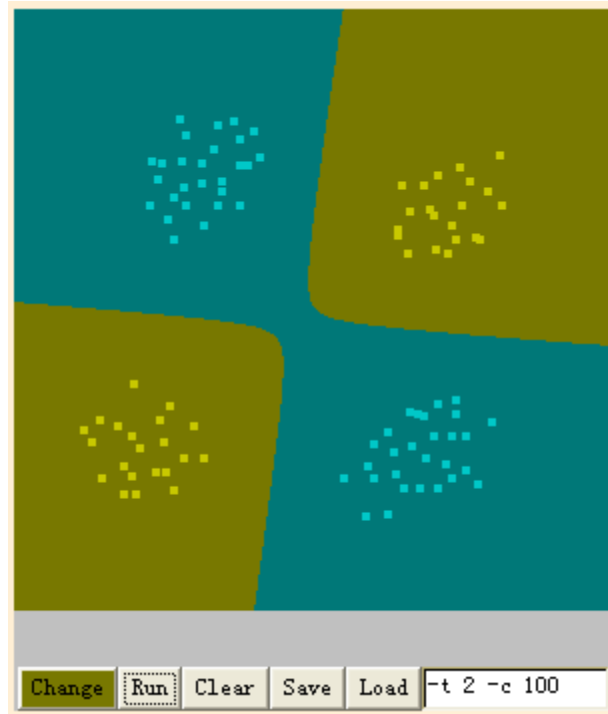
- which has zero empirical error on the XOR training set

Decision function defined by the SVM

- Notice that the decision boundaries are non-linear in the original space R^2 , but linear in the implicit space R^6

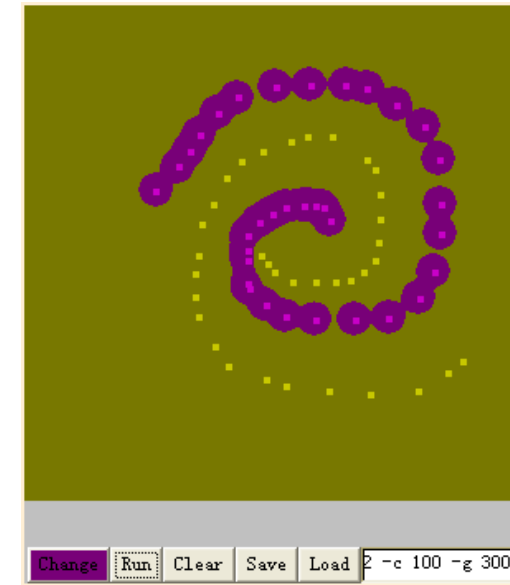
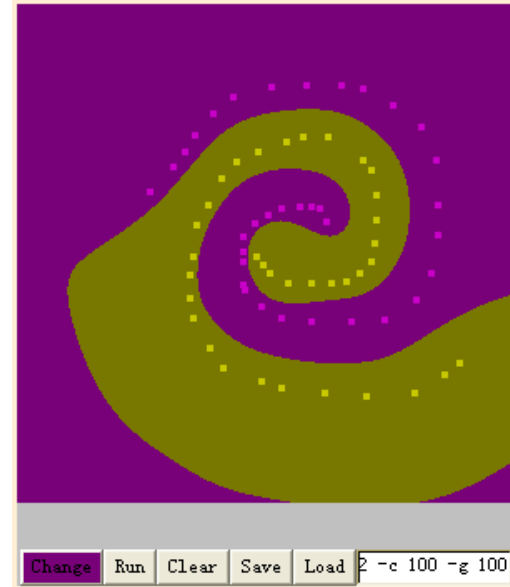


- “Radius basis functions” for the XOR problem



- Could solve complicated Non-Linear Problems
- γ and c control the complexity of decision boundary

$$\gamma = \frac{1}{\sigma^2}$$



Discussion

Advantages of SVMs

- There are no local minima, because the solution is a QP problem
- The optimal solution can be found in polynomial time
- Few model parameters to select: the penalty term C , the kernel function and parameters (e.g., spread σ in the case of RBF kernels)
- Final results are stable and repeatable (e.g., no random initial weights)
- SVM solution is sparse; it only involves the support vectors
- SVMs represent a general methodology for many PR problems: classification, regression, feature extraction, clustering, novelty detection, etc.
- SVMs rely on elegant and principled learning methods
- SVMs provide a method to control complexity independently of dimensionality
- SVMs have been shown (theoretically and empirically) to have excellent generalization capabilities

Challenges

- Do SVMs always perform best? Can they beat a hand-crafted solution for a particular problem?
- Do SVMs eliminate the model selection problem? Can the kernel functions be selected in a principled manner? SVMs still require selection of a few parameters, typically through cross-validation
- How does one incorporate domain knowledge? Currently this is performed through the selection of the kernel and the introduction of “artificial” examples
- How interpretable are the results provided by an SVM?
- What is the optimal data representation for SVM? What is the effect of feature weighting? How does an SVM handle categorical or missing features?